Advanced Engineering Informatics 76 (2026) 104973

Contents lists available at ScienceDirect

Advanced Engineering Informatics

journal homepage: www.elsevier.com/locate/aei

Full length article

Privacy-preserving distributed optimization for multi-UAV systems with threshold homomorphic encryption

Zhongyuan Zhao [a], Jihua Fang [a], Fu Zhang [a], Shuai Chen [a], Feng Xie [b], Hao Xu [a,*]

[a] College of Automation, Nanjing University of Information Science and Technology, Nanjing, 210044, China
[b] Jiangsu Automation Research Institute, Lianyungang, 222006, China

ARTICLE INFO

Keywords:
Privacy-preserving optimization
Homomorphic encryption
Threshold CKKS
Complex coordinate encryption
Distributed UAV systems

ABSTRACT

Multi-unmanned aerial vehicle (multi-UAV) systems have demonstrated considerable promise in applications such as large-scale logistics delivery and automated area inspection. However, distributed optimization requires frequent data exchange, which may expose sensitive state information. ~~To mitigate this critical issue~~, this paper introduces a privacy-preserving distributed optimization framework based on the threshold Cheon–Kim–Kim–Song (CKKS) homomorphic encryption scheme. The proposed framework seamlessly integrates the CKKS cryptosystem with distributed gradient-based optimization, thereby effectively shielding all participants' internal states and sensitive model coefficients from exposure. ~~Furthermore~~, a threshold decryption mechanism is incorporated to decentralize secret-key management and eliminate single points of trust, enhancing the system's robustness against semi-honest adversaries and resistance to collusion among up to $t-1$ parties. ~~Notably~~, the framework exploits the native support for approximate arithmetic in CKKS to enables efficient encryption of UAV coordinate data with provably bounded approximation error, leveraging their inherent complex-valued representation. Extensive experimental evaluations confirm that the framework attains convergence accuracy virtually indistinguishable from its non-encrypted (plaintext) counterpart while delivering verifiable semantic security guarantees. The proposed framework provides a privacy-preserving distributed optimization approach for multi-UAV systems.

1. Introduction

Multi-unmanned aerial vehicle (multi-UAV) systems have emerged as a transformative technology for a range of modern industrial applications, including large-scale logistics delivery, automated area inspection, and cooperative environmental sensing. By operating in a coordinated manner, UAV swarms can substantially enhance overall mission efficiency, spatial coverage, and systemic robustness [1–3]. ~~In these systems,~~ achieving precise cooperative positioning and maintaining stable formations are critical enablers for executing complex tasks. These requirements are commonly formulated as a distributed optimization problem, where each UAV iteratively minimizes a global objective function that encapsulates targets such as trajectory tracking and relative formation constraints through localized computation and communication with its neighbors [4–6]. ~~However~~, this distributed computational paradigm necessitates the frequent exchange of sensitive intermediate data, such as real-time states and gradient information, thereby raising significant privacy concerns [7–9]. ~~If critical data~~, including instantaneous positions, mission targets, and formation vectors, are intercepted by malicious participants or semi-trusted system operators, it could lead to the exposure of commercial secrets (e.g., mission strategies and delivery routes) or even precipitate targeted physical attacks, such as hijacking. ~~As highlighted in recent studies~~, the rapid expansion of commercial UAV operations has introduced significant security and privacy challenges, with most existing drones lacking adequate protection mechanisms against such threats [10].

The algorithmic evolution for UAV optimization mirrors a clear trajectory from isolated single-agent control to sophisticated multi-agent cooperation. Foundational work by [11] advanced single-UAV path planning by proposing an enhanced ant colony optimization algorithm that improves convergence speed and path quality in complex 3D environments. The constrained real-time optimization paradigm is extended to aerial logistics in [12]. The work addresses a pivotal application-specific challenge by developing a computationally efficient, near time-optimal trajectory optimization method. This method achieves over 92% reduction in computation time while sacrificing only minimal flight time. Consequently, it directly tackles the operational endurance constraints of delivery drones in complex urban environments. As research progressed to more complex operational environments, the work in [13] addressed the limitations of classical

* Corresponding author.
E-mail address: xuhao182@126.com (H. Xu).

https://doi.org/10.1016/j.aei.2026.104973
Received 16 December 2025; Received in revised form 14 March 2026; Accepted 13 June 2026
Available online 20 June 2026
1474-0346/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

approaches in dense urban topologies. The authors developed a novel hybrid metaheuristic framework (GS-PSO) that sequentially integrates particle swarm optimization, sparrow search algorithm, and genetic algorithm. Their contribution remains within the single-UAV framework. It enables enhanced exploration through chaotic mapping and adaptive learning mechanisms, thereby effectively balancing optimality and computational feasibility in highly intricate 3D urban scenarios. A natural progression from static environment planning was the incorporation of dynamic uncertainty. This challenge was addressed by [14], who advanced the state-of-the-art by integrating deep reinforcement learning with adaptive nonlinear model predictive control (ANMPC) for online trajectory replanning. Unlike conventional MPC with fixed weights, their approach dynamically optimizes the cost function's weight matrix in real time based on environmental changes. This enables a single UAV to adapt continuously to unforeseen disturbances while preserving short-term stability. This enhances system robustness far beyond the capabilities of static planners like that in [13], which is not adjust to dynamic uncertainties. Despite their individual advantages, these single-agent optimization schemes have an inherent limitation. They cannot effectively handle inter-agent coupling constraints or collaborative objectives. Such limitations are critical in multi-UAV systems, where tasks such as formation control and collision avoidance require coordinated decision-making.

This fundamental limitation of single-agent approaches precipitated a decisive shift in research focus toward multi-UAV cooperative optimization. The work in [15] marked a significant departure from the isolated agent paradigm. The authors devised a distributed control framework inspired by pigeon flock behavior. This framework enables a swarm to achieve and maintain desired geometric formations while simultaneously performing dynamic obstacle avoidance and collision avoidance. This demonstrated that complex global objectives could be effectively decomposed into local interaction rules executable by individual agents. Building upon the principle of decentralized decision-making, formalized the application of consensus-based optimization to multi-agent systems, providing a rigorous mathematical substrate for coordinating distributed tasks such as collaborative path planning and resource allocation in UAV swarms [16]. This line of inquiry evolved from specific algorithm design to broader synthesis, as exemplified by [17], which offered a comprehensive taxonomy of emergent swarm behaviors and control paradigms, thereby systematically mapping the landscape of collective intelligence achievable through decentralized coordination.

Progressing from foundational theory to practical deployment considerations, subsequent research began addressing non-ideal real-world constraints. Notably, the work in [18] tackled a critical practical impediment by developing distributed optimization algorithms that maintain provable convergence under asynchronous and delayed information exchanges, thereby enhancing the robustness of cooperative systems against imperfect communication links. While this collective body of work has substantially advanced the theoretical and practical understanding of multi-agent cooperation, a pervasive and critical assumption remains largely unexamined: the implicit trust model. Predominantly, these frameworks assume all participating agents and the central coordinator (if any) are fully trustworthy. This oversight neglects the acute privacy risks and potential for semi-honest (or even Byzantine) behaviors in practical, adversarial, or commercially sensitive environments, leaving a significant gap in the development of secure and privacy-preserving cooperative autonomy frameworks. To mitigate privacy leakage in distributed computation, existing methodologies primarily follow two distinct technical paths: cryptographic encryption and data perturbation. The differential privacy (DP) paradigm, pioneered by [19] and extended in [20], provides strong statistical privacy guarantees by injecting carefully calibrated noise (e.g., Gaussian or Laplacian) into shared data, such as gradients or state vectors. However, this injected noise inherently introduces a trade-off, invariably degrading the convergence accuracy and stability of the underlying optimization process-a compromise often unacceptable for UAV applications demanding high-precision coordination, such as tight formation flight or precision logistics.

The alternative cryptographic path employs homomorphic encryption (HE), which enables direct computation on encrypted data without decryption. Early schemes like Paillier encryption [21], supporting additive homomorphism, have been applied to protect basic distributed operations like average consensus. ~~However~~, its restriction to integer arithmetic necessitates complex fixed-point emulation for continuous-valued coordinates (e.g., UAV positions), leading to non-negligible precision loss and substantial computational overhead. While fully homomorphic encryption (FHE) schemes like BFV [22] support richer operations, they are also fundamentally designed for exact integer arithmetic, inheriting similar limitations when handling floating-point data prevalent in robotic systems. ~~In contrast~~, the CKKS scheme [23] supports approximate arithmetic on real and complex numbers. Its direct support for real and complex number operations offers a transformative advantage for robotics applications: UAV positional coordinates (e.g., a 2D location) can be efficiently encoded into complex numbers (mapping coordinates to real and imaginary parts) with high numerical fidelity and encrypted directly. This eliminates cumbersome and lossy data conversions, providing a cryptographically secure foundation for high-fidelity, privacy-preserving distributed optimization. However, traditional HE deployments typically rely on a single key holder for decryption, which creates a centralized point of trust and exposes the system to critical failure upon key compromise. This vulnerability can be addressed by threshold cryptography [24], which distributes decryption authority across multiple participants, requiring a predefined threshold of parties to collaborate for decryption. This mechanism effectively eliminates the single point of failure, aligns with the decentralized ethos of multi-UAV systems, and significantly elevates the system's security boundary. The inherent synergy between the distributed trust model of threshold cryptography and the approximate computation capability of CKKS presents a compelling yet underexplored avenue for constructing robust, privacy-preserving distributed optimization frameworks. The critical question of how to effectively integrate these two powerful primitives into a cohesive, efficient, and secure system for multi-UAV cooperation remains open. Existing threshold homomorphic encryption frameworks mainly focus on federated learning or generic secure computation [25,26]. Their interaction with iterative distributed optimization dynamics in multi-UAV systems has not been systematically investigated. In particular, the impact of approximate homomorphic encryption noise on the convergence behavior of distributed optimization algorithms remains insufficiently understood.

Motivated by these challenges, this paper proposes a Threshold CKKS-based Framework for Privacy-Preserving Distributed Optimization (TCKKS-PPDO) for multi-UAV systems. The proposed framework integrates the CKKS fully homomorphic encryption scheme with a threshold decryption mechanism to establish a secure and decentralized computational environment for distributed cooperative optimization. In this framework, UAV agents exchange encrypted states and perform homomorphic gradient computation while preserving the privacy of local states and sensitive optimization coefficients against semi-honest adversaries. Meanwhile, the threshold decryption mechanism eliminates the single point of trust in conventional homomorphic encryption systems and enhances resistance to collusion among participants. The main contributions of this paper are summarized as follows.

1. **Privacy-preserving distributed optimization framework for multi-UAV systems.** A threshold CKKS-based privacy-preserving distributed optimization framework is developed for multi-UAV cooperative positioning. The proposed framework integrates homomorphic encryption into the distributed optimization pipeline, enabling UAV agents to exchange encrypted states and perform secure gradient computation while preserving the decentralized interaction structure of multi-agent optimization.

2. **Noise-aware homomorphic computation and convergence characterization.** A homomorphic computation structure tailored for distributed quadratic optimization is designed, which requires only homomorphic additions and plaintext–ciphertext multiplications. This design avoids ciphertext–ciphertext multiplications that typically cause rapid noise growth in fully homomorphic encryption. ~~Based on this structure~~, the impact of CKKS approximation noise on the optimization process is theoretically characterized, and it is shown that the encrypted algorithm converges to a neighborhood of the optimal solution with a bounded error.
3. **Threshold-based decentralized decryption mechanism for secure multi-agent computation.** A (t, n)-threshold decryption protocol is incorporated into the distributed optimization process to eliminate the single point of trust in conventional homomorphic encryption systems. The secret key is shared among UAV agents, and plaintext gradients can only be reconstructed when at least t participants collaborate, thereby providing resistance to collusion and enhancing the security of the proposed framework.

The paper is structured as follows. Section 2 formulates the distributed optimization problem, defines the privacy threat model, and reviews the cryptographic foundations of the CKKS scheme and threshold decryption. Section 3 presents the design of the proposed threshold CKKS-based privacy-preserving optimization framework. Section 4 provides a formal analysis of algorithmic correctness and security. Experimental results evaluating computational efficiency, convergence accuracy, and privacy guarantees are reported in Section 5. Finally, Section 6 concludes the paper.

2. Preliminaries and problem formulation

This section begins by formalizing the gradient-based distributed optimization problem under consideration. ~~Subsequently,~~ the associated privacy vulnerabilities are analyzed, and the specific privacy definition adopted throughout this work is rigorously established.

2.1. Problem formulation

This paper studies the problem of privacy-preserving distributed position optimization for a swarm of N UAVs operating in a two-dimensional space. In the adopted system model, the position of each UAV is encoded as a complex number, $\mathbf{p}_i = x_i + jy_i \in \mathbb{C}$, where the real and imaginary components correspond to its X and Y Cartesian coordinates, respectively (see Table 1).

Assumption 1. The system operator (SO) is equipped with an undirected communication link to each agent i in the set $\mathcal{V}$.

The optimization objective function combines cooperative positioning with formation maintenance, given by

$$\min_{z_1,\dots,z_N} J = \frac{1}{2}\sum_{i=1}^{N}\|z_i - t_i\|^2 + \frac{\lambda}{2}\sum_{i=1}^{N}\sum_{j\in\mathcal{N}_i}\|z_i - z_j - d_{ij}\|^2. \quad (1)$$

目标追踪 编队保持

The objective function consists of a target tracking term and a formation maintenance term. The target tracking term guides the UAVs towards their designated positions, and the formation maintenance term enforces the desired geometric configuration among the swarm. A weighting coefficient λ balances the relative priority between these competing objectives.

The gradient is derived analytically as

$$\nabla_{z_i} J = (z_i - t_i) + \lambda \sum_{j\in\mathcal{N}i}(z_i - z_j - dij). \quad (2)$$

Table 1
Notation table.

Symbol	Description
N	Number of UAVs
$\mathcal{V}$	Set of all UAVs
i, j	UAV indices
$\mathcal{N}_i$	Set of neighboring UAVs of UAV i
n_i	Dimension of UAV i's state
ℓ, v	Component index within a vector
z_i	Position of UAV i, $z_i = x_i + jy_i$
x_i, y_i	Real (X) and imaginary (Y) coordinates of UAV i
$z_{i\ell}$	ℓth component of UAV i's state
$\mathbf{z} = [z_1, \dots, z_N]^\top$	Global state vector of all UAVs
Z_i	Feasible set for UAV i's position
t_i	Target position of UAV i
d_{ij}	Desired relative formation vector between UAV i and j
λ	Formation maintenance weight coefficient
$J(\mathbf{z})$	Global objective function
$\nabla_{z_i} J$	Gradient of J with respect to z_i
$\boldsymbol{\Phi}_i(\mathbf{z})$	Gradient component for UAV i
$\boldsymbol{\Phi}_{i\ell}$	ℓth component of $\boldsymbol{\Phi}_i$
A	System matrix in gradient expression
B	Constant vector in gradient expression
$A_{i\ell}^{jv}$	Coefficient of z_{jv} in $\boldsymbol{\Phi}_{i\ell}$
$B_{i\ell}$	Constant term in $\boldsymbol{\Phi}_{i\ell}$
ζ	Iteration counter
$\eta(\zeta)$	Step size at iteration ζ
$\mathcal{P}_{Z_i}(\cdot)$	Projection operator onto feasible set Z_i
λ	Security parameter
N	Polynomial degree
q	Ciphertext modulus
q_ℓ	Modulus at level ℓ
Δ	Scaling factor
χ	Discrete Gaussian noise distribution
R_q	Polynomial ring modulo q
m	Plaintext polynomial obtained after encoding
$m_{i\ell}(\zeta)$	Encoded polynomial corresponding to the ℓth component of UAV i's state at iteration ζ
$\hat{m}$	Ciphertext encrypting a message
$\hat{m}_{i\ell}^{j}(\zeta)$	Ciphertext sent from UAV i to UAV j (or SO) at iteration ζ
$\boldsymbol{\Phi}_{i\ell}(z(\zeta))$	Plaintext gradient component ℓ for UAV i at iteration ζ
$\hat{\boldsymbol{\Phi}}_{i\ell}^{s}(\zeta)$	Encrypted gradient component ℓ for UAV i at iteration ζ
$\boldsymbol{\Phi}_{i\ell}^{s}(\zeta)$	Plaintext polynomial of gradient component ℓ (before decoding) for UAV i at iteration ζ
$\boldsymbol{\Phi}_{i\ell}(\zeta)$	Decrypted and decoded gradient component ℓ for UAV i at iteration ζ
$\boldsymbol{\Phi}_i(\zeta)$	Decrypted and decoded gradient for UAV i at iteration ζ
t	Threshold number of parties required for decryption
n	Total number of parties (UAVs)
pk	Public key (shared by all parties)
sk_i	Secret key share held by party i
s	Master secret key (shared among parties)
λ_i^S	Lagrange coefficient for party i in reconstruction set S
pt_j	Partial decryption from party j
e, e_1, e_2, u	Random noise polynomials sampled from χ
η_{enc}	Noise from encryption
η_{add}	Noise from homomorphic addition
η_{scalar}	Noise from plaintext-ciphertext multiplication
η_{hom}	Total noise from homomorphic operations
η_{thresh}	Noise from threshold decryption
η_{total}	Total accumulated noise
η_{bound}	Maximum allowable noise for correct decryption
ϵ_{enc}	Bound on gradient noise
L_A	Maximum absolute value of coefficients in A
L_z	Maximum absolute value of state variables
L_B	Maximum absolute value of constants in B
σ	Number of precision bits preserved
κ	Depth-dependent constant for noise growth
$\text{VIEW}_i^{\Pi}(y)$	View of party i during protocol Π with input y
S	Simulator for security proof
$\text{negl}(\lambda)$	Negligible function in security parameter λ
$\mathbf{Adv}_D(k)$	Advantage of distinguisher D

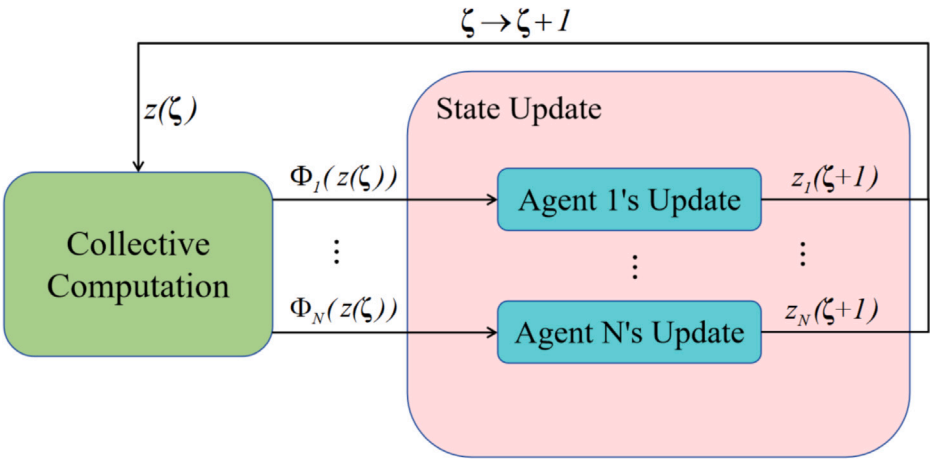


Fig. 1. Block diagram of the computation problem.

To enable homomorphic encryption, the gradient is expressed in matrix form:

$$\nabla J = A\mathbf{z} + B, \tag{3}$$

where $\mathbf{z} = [z_1, z_2, \ldots, z_N]^\top \in \mathbb{C}^N$ denotes the position vector, $A \in \mathbb{C}^{N\times N}$ is the system matrix, and $B \in \mathbb{C}^N$ is a constant vector.

The UAVs address this distributed optimization problem via projected gradient methods. For agent i, its state is updated iteratively as follows

$$z_i(\zeta+1) = \mathbb{P}_{Z_i}\left[z_i(\zeta) - \eta(\zeta)\boldsymbol{\Phi}_i(z(\zeta))\right], \tag{4}$$

where ζ represents the discrete time step, $\eta(\zeta) > 0$ is the step size for the ζth update, and $\boldsymbol{\Phi}_i$ denotes the first-order gradient of the objective function (1) with respect to z_i given in (3). The gradient $\boldsymbol{\Phi}_i$ is typically a function of the entire global state vector $\mathbf{z}$. At each step ζ, the agents update their states via (4) with the objective of converging to a solution of the distributed optimization problem.

2.2. Privacy definition

Within distributed optimization frameworks, privacy boundaries among participants $\mathcal{V}\cup\mathrm{SO}$ are defined by three critical classes of data: the state sequences $z_i(\zeta)$ and feasible sets Z_i, which are held exclusively by each agent $i \in \mathcal{V}$, and distributed subsets of objective function coefficients $\{C^i_{\boldsymbol{\Phi}}\}_{i\in\mathcal{V}}\cup\{\mathrm{SO}\}$. The coefficients satisfy the global coverage condition $\bigcup_i C\boldsymbol{\Phi}^i = C_{\boldsymbol{\Phi}}$, with each coefficient $r \in C_{\boldsymbol{\Phi}}$ accessible only to its designated holders. Although the system operator (SO) possesses knowledge of the analytical structure of $\boldsymbol{\Phi}_i$ and its own coefficients $C^{\mathrm{SO}}_{\boldsymbol{\Phi}}$, it cannot access real-time states $z(\zeta)$ or private coefficients held by other participants $\bigcup_{j\neq\mathrm{SO}} C^j_{\boldsymbol{\Phi}}$. This information asymmetry arises from stringent privacy constraints: disclosure of states may expose behavioral patterns, while leakage of sensitive coefficients could induce systemic risks.

As illustrated in Fig. 1, the projected gradient update in (4) proceeds in two decoupled phases. ~~First~~, during the collective computation phase, participants collaboratively compute gradient values $\boldsymbol{\Phi}_i(z(\zeta))$, necessitating the exchange of private data across entities. This introduces a secure multiparty computation (SMC) challenge: ensuring that agent i correctly obtains $\boldsymbol{\Phi}_i(z(\zeta))$ while preventing the inference of other participants' states $\{z_j(\zeta)\}j \neq i$ or private coefficients $C^j_{\boldsymbol{\Phi}}$. Subsequently, in the local update phase, each agent independently performs the projection operation $\mathbb{P}_{Z_i}[\cdot]$ without further interaction.

This work addresses the presence of semi-honest (honest-but-curious) adversaries [27], which may corrupt a subset of agents within the system. Each corrupted agent $i \in \mathcal{V}\cup\{\mathrm{SO}\}$ adheres to the prescribed protocol while simultaneously exploiting information acquired during execution to infer the private data of other participants. ~~Moreover~~, such corrupted agents may collude by exchanging their respective views to jointly infer confidential information. This work adopts the semi-honest adversary model. In this model, corrupted agents follow the protocol but attempt to infer private information. In a malicious model, adversaries may arbitrarily deviate from the protocol specification, such as by injecting false data or refusing to participate, which could disrupt the optimization process or lead to incorrect results. Designing efficient protocols secure against malicious adversaries typically necessitates the use of costly cryptographic primitives like zero-knowledge proofs (ZKPs) for verifiable computation [28]. However, for resource-constrained multi-UAV systems where computational efficiency and low communication overhead are paramount, such additions could impose prohibitive burdens. The semi-honest model remains a pragmatic and widely-adopted foundation for privacy-preserving distributed systems, effectively mitigating prevalent risks such as data eavesdropping by the system operator or inference attacks by colluding UAV nodes. ~~Therefore,~~ this work focuses on establishing a robust and efficient framework under the semi-honest assumption, leaving resilience against active attacks as a valuable direction for future research. Furthermore, corrupted agents may collude by exchanging their respective protocol views to jointly deduce confidential information. The formal definition of secure multiparty computation (SMC) privacy adopted in this work is subsequently presented, followed by its elaboration within the standard view-based framework commonly utilized in cryptographic analysis.

Suppose a system comprises M distinct parties. Every participant i holds a confidential dataset $y_i \in \mathbb{C}^{d_i}$. The global dataset is formed by concatenating all parties' data into a vector $y = [y_i] \in \mathbb{C}^d$ with total dimension $d = \sum_{i=1}^M d_i$. Each party i requires computation of specific functions $h_i : \mathbb{C}^d \to \mathbb{C}^{r_i}$ that depend on the complete dataset y. The collective computation target is defined as $h = [h_i] : \mathbb{C}^d \to \mathbb{C}^r$ where the output dimension aggregates as $r = \sum_{i=1}^M r_i$. Let Π denote a protocol enabling these M parties to jointly compute h. Each party i provides its private data y_i as input to Π. During protocol execution, party i receives t_i messages, denoted $v^i_1, \ldots, v^i_{t_i}$. The complete information observable to party i during Π's execution with input y is termed its *view*, defined formally as $\mathrm{VIEW}^{\Pi}_i(y) \triangleq \{y_i, v^i_1, \ldots, v^i_{t_i}\}$. To establish a rigorous privacy framework for secure multiparty computation (SMC), we begin by presenting the foundational notion of computational indistinguishability.

Definition 1. Let $\mathscr{X} = \{\mathscr{X}_k\}_{k\in\mathbb{N}}$ and $\mathscr{Y} = \{\mathscr{Y}_k\}_{k\in\mathbb{N}}$ denote two distribution ensembles, where each $\mathscr{X}_k$ and $\mathscr{Y}_k$ are random variables over a common range $\mathcal{R}_k$. We say $\mathscr{X}$ and $\mathscr{Y}$ are computationally indistinguishable, denoted $\mathscr{X} \overset{c}{\equiv} \mathscr{Y}$, if for every non-uniform probabilistic

polynomial-time (PPT) distinguisher D, and every positive polynomial $p(\cdot)$, there exists an integer N such that for all $k > N$, it holds that

$$\left|\Pr\left[D(\sigma_x) = 1\right] - \Pr\left[D(\sigma_y) = 1\right]\right| \leq \frac{1}{p(k)}, \tag{5}$$

Equivalently, the distinguishing advantage satisfies

$$\mathbf{Adv}_D(k) = \left|\Pr[D(\mathscr{X}_k) = 1] - \Pr[D(\mathscr{Y}_k) = 1]\right| = \mathrm{negl}(k), \tag{6}$$

where $\mathrm{negl}(k)$ represents a negligible function in k.

Definition 1 provides rigorous mathematical foundation for cryptographic security by formalizing the adversary's distinguishing advantage $\mathbf{Adv}_D(k)$ between real and ideal distributions. This formulation transforms the intuitive notion of "unbreakability" into a verifiable criterion, requiring the advantage to be negligible ($\mathrm{negl}(k)$). Critically, this framework directly underpins the SMC privacy definition (Definition 2) and enables security proofs for homomorphic encryption schemes. The fundamental implication is that if adversaries cannot distinguish actual protocol executions from simulator-generated computation traces, the protocol can be proven to leak no extraneous private information.

Definition 2. A protocol Π for computing functions $h = (h_1, \ldots, h_M)$ is said to *securely compute* h in the presence of semi-honest adversaries if for every party $i \in \{1, \ldots, M\}$ and all input tuples $y = (y_1, \ldots, y_M)$, one can construct a probabilistic polynomial-time (PPT) simulator $\mathcal{S}$ satisfying

$$\mathcal{S}\left(i, y_i, h_i(y)\right) \overset{c}{\equiv} \mathrm{VIEW}_i^{\Pi}(y), \tag{7}$$

where

- $\mathrm{VIEW}_i^{\Pi}(y)$ indicates the entire information observable to party i during protocol execution.
- $\overset{c}{\equiv}$ indicates computational indistinguishability (Definition 1).
- $\mathcal{S}$ generates simulated transcripts using only i's input y_i and output $h_i(y)$.

This condition ensures that the protocol reveals nothing beyond the prescribed outputs to any participant.

Definition 2 guarantees that in a secure multi-party computation (SMC) protocol, each participant learns nothing beyond its own input y_i and its designated output $h_i(y)$. Formally, this guarantee is provided by the existence of a probabilistic polynomial-time simulator $\mathcal{S}$ that, given only $y_i, h_i(y)$, can generate a view that is computationally indistinguishable from the participant's actual view during a real protocol execution. This simulation-based condition proves that the protocol reveals no additional information beyond what can be derived from the designated input–output pairs.

To address privacy challenges in SMC for multi-agent systems, this work adopts a homomorphic encryption (HE) approach. Within this framework, agents first encrypt their private data and transmit the resulting ciphertexts to a potentially semi-honest system operator. As the central computational entity, the operator directly evaluates the predefined functions Φ_i on the received ciphertexts and returns the encrypted results to the agents. Through local decryption of these results, the agents obtain the output values of Φ_i. This workflow ensures that agents access only aggregated outputs without ever being exposed to raw individual data, thereby resolving external privacy concerns in SMC through a synergistic combination of ciphertext computation and distributed decryption.

Although the described architecture addresses external privacy issues, protection against potential inference of raw data from ciphertexts by semi-honest operators remains essential. Accordingly, following the established privacy definitions in the homomorphic encryption literature [29,30], this work employs a dual-security mechanism:

1. **Plaintext non-recoverability (computational security):** Adversaries cannot recover plaintexts from ciphertexts using any polynomial-time algorithm, with security relying on established computational hardness assumptions. In the CKKS scheme, this property is realized through the computational intractability of the ring learning with errors (RLWE) problem, preventing operators from efficiently recovering floating-point plaintexts. Security validation relies on empirical assessment of known super-polynomial-time attacks.
2. **Plaintext indistinguishability (semantic security):** Adversaries cannot distinguish between ciphertexts of arbitrary plaintexts with non-negligible advantage. The CKKS scheme satisfies this through indistinguishability under chosen-plaintext attack (IND-CPA), ensuring that even when an operator adaptively chooses floating-point plaintext pairs, its success probability in identifying the encrypted source remains negligibly close to 0.5. Notably, while indistinguishability strictly implies non-recoverability, the converse does not necessarily hold; this limitation is mitigated by CKKS's approximate noise characteristics, which enhance statistical privacy protection in floating-point computations beyond what integer-based schemes (e.g., Paillier) can offer.

The standard formalization of this notion is semantic security, defined as follows.

Definition 3. Suppose there exists an encryption scheme F whose encryption process is $z \leftarrow F(z)$. The adversary $\mathcal{A}$ selects a pair of plaintexts (z_0, z_1) and requests the scheme holder to provide ciphertexts, who computes $c_0 \leftarrow F(z_0)$ and $c_1 \leftarrow F(z_1)$, then sends the unordered pair $\{c_0, c_1\}$ to $\mathcal{A}$ without revealing the correspondence between ciphertexts and plaintexts. The scheme F is semantically secure if for all probabilistic polynomial-time adversaries $\mathcal{A}$, the probability that $\mathcal{A}$ correctly identifies which ciphertext corresponds to which plaintext is at most $\frac{1}{2} + \mathsf{negl}(\lambda)$, i.e., $c_0 \overset{c}{\equiv} c_1$, where λ is the security parameter and negl is a negligible function.

Definition 3 establishes a fundamental framework for provably secure encryption schemes. Semantic security is typically demonstrated via a reductionist approach: by assuming the computational hardness of well-studied mathematical problems (e.g., integer factorization, discrete logarithm, or ring learning with errors), it is shown that any polynomial-time adversary capable of distinguishing ciphertexts with non-negligible advantage could be transformed into an efficient algorithm that solves the underlying hard problem. This reduction paradigm has enabled the rigorous security analysis of a wide range of encryption schemes, including Goldwasser-Micali, ElGamal, Paillier, Boneh–Goh–Nissim, and modern lattice-based constructions such as the CKKS fully homomorphic encryption scheme.

2.3. CKKS encryption scheme

The CKKS (Cheon–Kim–Kim–Song) homomorphic encryption scheme [23] is adopted as the cryptographic foundation. This choice is motivated by CKKS being an efficiently implementable fully homomorphic encryption scheme that supports approximate arithmetic over real and complex numbers. In a comparative assessment of public-key encryption schemes, CKKS exhibits distinct technical advantages over alternatives such as Paillier and BFV (Brakerski–Fan–Vercauteren), particularly for processing continuous-valued data. Schemes like Paillier and BFV are inherently designed for exact integer arithmetic, necessitating explicit preprocessing — such as scaling and rounding — to encode real numbers into the integer domain. These conversions introduce substantial computational overhead and may incur cumulative precision loss. In contrast, CKKS natively supports encryption over the complex field, where real-valued inputs are seamlessly embedded as

complex numbers with zero imaginary parts. This intrinsic feature eliminates the need for explicit data transformation, thereby simplifying the algorithmic pipeline and avoiding precision degradation. Although CKKS introduces controlled approximation errors upon decryption, its built-in scaling mechanisms and noise management techniques maintain high numerical accuracy within predefined error bounds. This balance between efficiency and precision renders CKKS particularly suitable for applications requiring complex-number arithmetic, including privacy-preserving machine learning and signal processing. Overall, CKKS surpasses integer-centric schemes by offering native support for complex data, a streamlined workflow, and tunable error control, establishing it as a preferred choice for modern cryptographic applications that demand both flexibility and computational fidelity. The CKKS encryption scheme consists of four fundamental components: parameter selection, key generation, encoding algorithm, encryption algorithm, decryption algorithm and decoding algorithm. The detailed processes for each component are outlined in Algorithm 1.

Algorithm 1: ~~CKKS Encryption Scheme~~

Parameter Selection:
1: Select security parameter λ, polynomial degree N.
2: Choose base modulus q_0, define the modulus chain that

$$q_\ell = q_0 \cdot p^\ell \quad (\text{for } \ell = 0, 1, \ldots, L).$$

3: Set scaling factor Δ .
4: Select the discrete Gaussian distribution χ to generate noise polynomials.

Key Generation(s, pk):
1: SecretKeyGen: Sample $s \leftarrow \chi$ from R_{q_L}.
2: PublicKeyGen:

- Sample $a \leftarrow R_{q_L}$, $e \leftarrow \chi$.
- Compute $b = [-a \cdot s + e]_{q_L}$.
- Output $\text{pk} = (b, a) \in R_{q_L}^2$.

Encoding ($\mathbf{z}, \Delta$):
1: **Input**: Complex vector $\mathbf{z} = (z_j)_{j \in \mathcal{V}} \in \mathbb{C}^{N/2}$ and Δ.
2: Expand $\mathbf{z}$ to conjugate-symmetric vector $\mathbf{z}' \in \mathbb{H} \subset \mathbb{C}^N$.
3: Apply inverse canonical embedding and construct polynomials:

$$\hat{m} = \sigma^{-1}(\mathbf{z}') \in \mathbb{C}[X]/(X^N + 1).$$

4: Scale and discretize:

$$m = \lfloor \Delta \cdot \hat{m} \rceil \in R, R = \mathbb{Z}[X]/(X^N + 1).$$

5: **Output**: Polynomial $m(X) \in R$.

Encryption ($\mathcal{E}_{\mathbf{pk}}(m)$):
1: **Input**: Encoded polynomial $m \in R$.
2: Sample $u, e_1, e_2 \leftarrow \chi$.
3: Compute ciphertext $\hat{m} = (c_0, c_1)$:

$$\begin{cases} c_0 = [b \cdot u + e_1 + m]_{q_L}, \\ c_1 = [a \cdot u + e_2]_{q_L}. \end{cases}$$

4: **Output**: $\hat{m} = (c_0, c_1) \in R_{q_L}^2$.

Decryption ($D_{\mathbf{sk}}(\hat{m})$):
1: **Input**: Ciphertext $\hat{m} = (c_0, c_1)$ at level ℓ.
2: Recover noisy message:

$$m' = [c_0 + c_1 \cdot s]_{q_\ell} \in R_{q_\ell}.$$

3: **Output**: $m' = m + e_{\text{total}}$.

Decoding (m', Δ):
1: **Input**: Decrypted polynomial $m' \in R_{q_\ell}$ and Δ.
2: Rescale: $\hat{m} = m'/\Delta \in \mathbb{C}[X]/(X^N + 1)$.
3: Apply canonical embedding: $\mathbf{v} = \sigma(\hat{m}) \in \mathbb{C}^N$.
4: Project to slots: $\mathbf{z}' = \pi(\mathbf{v}) \in \mathbb{C}^{N/2}$.
5: **Output**: Approximate vector $\mathbf{z}' \approx \mathbf{z}$.

In cryptography, the security of public-key encryption is typically built upon mathematical assumptions regarding computationally hard problems. A prime example is the semantic security of CKKS fully homomorphic encryption, which hinges on the classic Ring Learning With Errors (RLWE) assumption [31], which is formulated as follows: given a polynomial ring $R_q = \mathbb{Z}_q[X]/(X^N + 1)$ (where N is a power of 2 and q is a large modulus), and a set of samples of the form $(a_i, a_i \cdot S_i + e_i)$ (S_i is the secret key and e_i is the small error term), it is computationally infeasible to distinguish these samples from uniformly random samples over the ring R_q. This assumption ensures the semantic security that an attacker cannot recover the secret parameters from public information or decode the ciphertext.

2.4. Threshold-based homomorphic encryption extension

This section introduces threshold homomorphic encryption (THE), a cryptographic primitive that extends conventional homomorphic encryption to decentralized trust settings. By integrating secret sharing with fully homomorphic capabilities, THE facilitates secure multiparty computation in which no single party holds full decryption authority. A formal model of THE schemes is presented, followed by a concrete instantiation based on the CKKS cryptosystem. The description includes two core protocols: distributed key generation and threshold decryption.

Definition 4 (*Threshold Homomorphic Encryption Scheme*)**.** A (t, n)-threshold homomorphic encryption scheme over plaintext space $\mathcal{M}$ consists of the following five probabilistic polynomial-time algorithms.

- **DistKeyGen**$(1^\lambda, t, n) \rightarrow (pk, \{sk_i\}_{i=1}^n)$. Generates a public key pk and n secret shares $\{sk_i\}$ such that any t shares can collaboratively decrypt, while fewer than t shares reveal no information about the master secret.
- **Enc**$(pk, m) \rightarrow \hat{m}$. Encrypts message $m \in \mathcal{M}$ into ciphertext $\hat{m}$.
- **Eval**$(pk, f, \{\hat{m}_j\}) \rightarrow \hat{m}_{out}$. Homomorphically evaluates function f on ciphertexts.
- **PDec**$(sk_i, \hat{m}_{out}) \rightarrow pt_i$. Computes partial decryption pt_i using secret share sk_i.
- **Combine**$(\{pt_i\}_{i \in S}, \hat{m}_{out}) \rightarrow m'$. Combines $\geq t$ partial decryptions to reconstruct plaintext m'.

In a (t, n)-threshold homomorphic encryption scheme, the choice of the threshold t balances security against collusion and system availability. For our framework, we set

$$t = \left\lfloor \frac{n}{5} \right\rfloor + 1, \tag{8}$$

which ensures that any coalition of fewer than t nodes cannot recover the secret key or decrypt ciphertexts, while still allowing decryption as long as at least t nodes are online. This value is chosen to tolerate a moderate number of node failures (up to $n - t$) while keeping the collusion resistance sufficiently high for typical UAV swarm sizes.

2.4.1. Distributed key generation protocol

The distributed key generation (DKG) protocol establishes shared secrets without trusted parties, satisfying correctness, secrecy, and robustness properties. Algorithm 2 implements Feldman's verifiable secret sharing scheme (VSS) [32] adapted for RLWE-based schemes.

Algorithm 2: RLWE-based DKG for CKKS
$(pk, \{sk_i\}_{i=1}^n) \leftarrow \text{DKG}(1^\lambda, t, n)$

Require: Security parameter λ, threshold t, parties n
Ensure: Public key pk, secret shares $\{sk_i\}_{i=1}^n$
1: Each party i samples polynomial
$f_i(z) = s_{i0} + s_{i1} z + \cdots + s_{i(t-1)} z^{t-1} \in R_q$
2: Compute commitments: $c_{ik} = g^{s_{ik}} \mod p$ for $0 \leq k < t$
3: Broadcast $\mathbf{c}_i = (c_{i0}, \ldots, c_{i(t-1)})$ to all parties
4: Construct public polynomial: $F_k = \prod_{i=1}^n c_{ik} = g^{\sum_i s_{ik}} \mod p$
5: Set $pk = (F_0, \ldots, F_{t-1})$
6: **for** $j = 1$ to n **do**
7: Compute $sk_j = \sum_{i=1}^n f_i(j) \bmod q$
8: **end for**

Remark 1. Algorithm 2 is explicitly designed to operate within the algebraic framework of the CKKS scheme, which relies on the polynomial ring $R_q = \mathbb{Z}_q[X]/(X^N + 1)$ and the Ring Learning With Errors (RLWE) assumption. Each party i samples a random polynomial

$$f_i(z) = s_{i0} + s_{i1}z + \cdots + s_{i(t-1)}z^{t-1} \in R_q[z] \tag{9}$$

with coefficients in R_q. The master secret key is defined as the constant term of the aggregated polynomial $F(z) = \sum_i f_i(z)$, i.e., $s = \sum_i s_{i0} \in R_q$, and each party's secret share is $sk_j = \sum_i f_i(j) \in R_q$. The public key pk is then derived from s using the standard CKKS key generation (Algorithm 1), which is semantically secure under the RLWE assumption. The commitments $c_{ik} = g^{s_{ik}} \bmod p$ are computed in a multiplicative group where the discrete logarithm is hard; they serve only for verifiability and do not reveal any information about the shares, preserving the information-theoretic privacy of Shamir secret sharing. Thus, the DKG protocol seamlessly integrates with the algebraic structure of CKKS while maintaining the required security guarantees.

2.4.2. Threshold decryption protocol

The threshold decryption protocol enables collaborative plaintext recovery while preventing single-point key exposure. Algorithm 3 specifies the decryption procedure with robustness guarantees.

Algorithm 3: Threshold Decryption for CKKS
$m \leftarrow$ T-Dec$(\hat{m}, \{sk_i\}_{i\in S}, S)$

Require: Ciphertext $\hat{m} = (c_0, c_1)$, shares $\{sk_i\}$, combiner set S $(|S| \geq t)$
Ensure: Plaintext m
1: Each $i \in S$ computes partial decryption:
$pt_i = c_0 + c_1 \cdot sk_i + e_i \mod q$
2: Compute Lagrange coefficients: $\lambda_i^S = \prod_{j\in S\setminus\{i\}} \frac{j}{j-i}$
3: Recover noisy plaintext: $m' = \sum_{i\in S} \lambda_i^S \cdot pt_i \mod q$
4: Output $m =$ Decoding(m', Δ)

Remark 2. The threshold decryption in Algorithm 3 is tailored to the noise and ring structure of CKKS. Each partial decryption is computed as

$$pt_j = \hat{\Phi}^s_{i\ell 0} + \hat{\Phi}^s_{i\ell 1} \cdot sk_j + e_j, \tag{10}$$

where e_j is a small noise term sampled from the same discrete Gaussian distribution χ used in CKKS encryption. This noise guarantees that fewer than t partial decryptions reveal no information about the plaintext under the RLWE assumption. The reconstruction step combines at least t partial decryptions via Lagrange interpolation in R_q:

$$\Phi^s_{i\ell} = \sum_{i\in S} \lambda_i^S \cdot pt_i, \quad \text{with } \lambda_i^S = \prod_{j\in S\setminus\{i\}} \frac{j}{j-i} \in R_q, \tag{11}$$

where the inverses exist because the modulus q is chosen such that all denominators are invertible (e.g., q prime). This yields $\Phi^s_{i\ell} = \Delta \cdot \Phi_{i\ell}(z) + \eta_{\text{total}}$. Consequently, the threshold decryption correctly recovers the scaled gradient while preserving the noise characteristics essential for CKKS's approximate arithmetic and maintaining compatibility with the RLWE security foundation.

3. Privacy-preserving decentralized

This section presents the proposed privacy-preserving distributed optimization protocol for multi-UAV systems, which integrates the CKKS homomorphic encryption scheme with threshold decryption. The protocol enables secure collaborative computation of gradients while protecting each UAV's private state and model coefficients.

3.1. Algorithm design

The analysis focuses on the case where the gradients $\Phi_{i\ell}$ are affine functions, and introduces the Threshold CKKS-based Distributed Secure Computation Algorithm to achieve semantic security against a semi-honest system operator while maintaining resilience against persistent attacks. This section focuses on a defined set of computational problems, with their formal characterization given as embodied by the assumption below.

Assumption 2. For every agent $i \in \mathcal{V}$ and each index $\ell \in \{1, \ldots, n_i\}$, the function $\Phi_{i\ell}$ is affine with respect to the global state z, and all its coefficients are available to the system operator.

By Assumption 2, every $\Phi_{i\ell}$ can be expressed as a weighted sum of the state components z, with weights known to the operator. For notational simplicity, we represent each $\Phi_{i\ell}$ in the form

$$\Phi_{i\ell}(z) = \sum_{j=1}^{N} \sum_{v=1}^{n_j} A^{jv}_{i\ell} z_{jv} + B_{i\ell}, \tag{12}$$

where the coefficients $A^{jv}_{i\ell} \in \mathbb{C}$ and $B_{i\ell} \in \mathbb{C}$ are all known to the system operator. For each $j \in \mathcal{V}$, we define the row vector $A^j_{i\ell} = [A^{j1}_{i\ell}, \ldots, A^{jn_j}_{i\ell}]$, and further construct the matrix $A^j_i = [A^j_{i\ell}]_{\ell\in\{1,\ldots,n_i\}}$. For each agent $i \in \mathcal{V}$, we first define, for each $\ell = 1, \ldots, n_i$, the row vector $A_{i\ell} = [A^1_{i\ell}, \ldots, A^N_{i\ell}]$. Subsequently, we form the matrix $A_i = [A^1_i, \ldots, A^N_i]$ and the column vector $B_i = [B_{i1}, \ldots, B_{in_i}]^T$. Then, we define the global matrices $A = [A_i]_{i\in\mathcal{V}}$ and $B = [B_i]_{i\in\mathcal{V}}$. Using the notation above, we obtain the compact expressions: $\Phi_{i\ell}(z) = \sum_{j=1}^N A^j_{i\ell} z_j + B_{i\ell} = A_{i\ell} z + B_{i\ell}$ and $\Phi_i(z) = A_i z + B_i$.

The proposed Algorithm 4 is constructed based on the public-key CKKS homomorphic encryption scheme. The cryptographic preliminaries of the CKKS scheme are summarized in Algorithm 1.

Fig. 2 illustrates the complete data flow of the proposed protocol in one iteration, where dashed arrows (supplemented with lock icons) represent encrypted transmissions, and solid arrows represent plaintext requests or local operations. The detailed procedure is as follows: UAV i and other UAVs encode and encrypt their own states z_i, z_j into $\hat{z}_i, \hat{z}_j$ respectively, and send them to the system operator (SO); the SO performs homomorphic gradient computation over ciphertexts to obtain $\hat{\Phi}_i$, and returns the encrypted result to UAV i; then UAV i sends decryption requests to at least t other UAVs, which use their own secret shares sk_j to compute partial decryptions pt_j and return them; after collecting sufficiently many partial decryptions, UAV i combines them locally to obtain the plaintext gradient Φ_i, and executes a projected gradient descent update for its state. This figure intuitively demonstrates that all transmitted states and gradients remain encrypted, so the system operator cannot access any plaintext information. Moreover, the threshold decryption process only involves partial decryption interactions and requires at least t participants to recover the plaintext, thereby ensuring both correct execution of the algorithm and fulfillment of privacy requirements, while also addressing the reviewer's concern regarding the scope of encryption in the information flow.

In the distributed CKKS-based secure computation framework, both the correctness and the security of the algorithm critically depend on appropriate parameter selection. To guarantee correct decryption, the following assumption is required.

Assumption 3. For each agent i, the CKKS parameters shall satisfy the following conditions at any step ζ.

1. The magnitudes of coefficients and states are bounded by

$$\|A_i\|_{\max} \leq L_A, \quad \|z(\zeta)\|_{\max} \leq L_z, \quad \|B_i\|_{\max} \leq L_B, \tag{13}$$

where L_A, L_z, L_B are predefined constants.

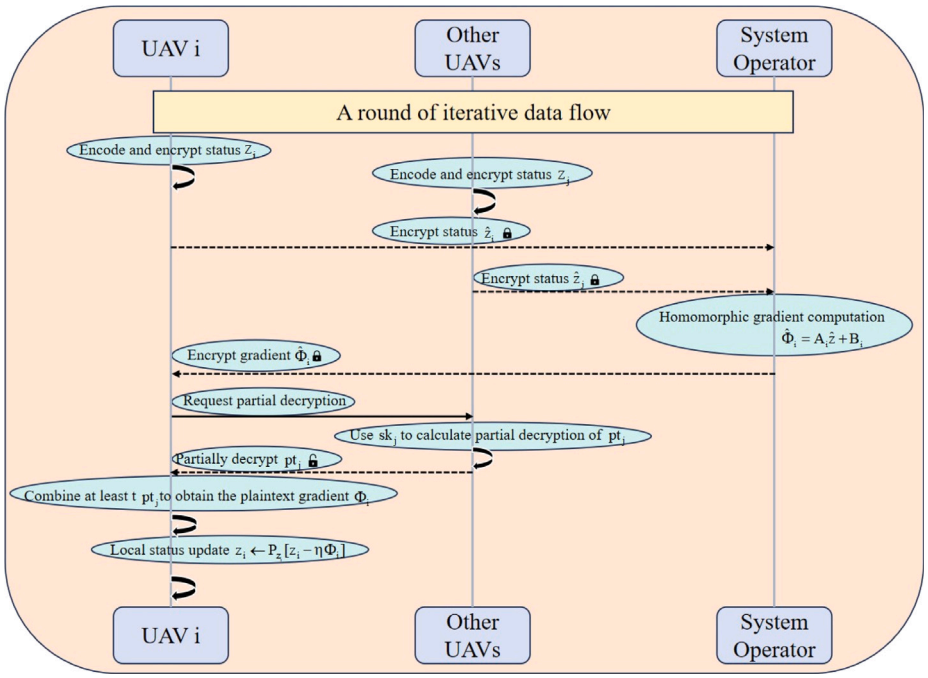


Fig. 2. Data flow of the proposed protocol in one iteration.

2. The scaling factor Δ satisfies

$$\Delta \geq 10^{\sigma} \cdot \left(L_A \cdot L_z + L_B\right), \tag{14}$$

where σ is the precision bit-width to preserve significant digits.

3. The ciphertext modulus q is chosen such that

$$q \geq 2\Delta \cdot \left(L_A \cdot L_z + L_B\right) + \kappa \cdot \eta_{\max}, \tag{15}$$

where κ is a depth-dependent constant and $\eta_{\max}$ bounds single-operation noise growth.

4. Denote $\eta_{\text{bound}} = 2\Delta \cdot 10^{-\sigma}$. The total noise $\eta_i(\zeta)$ after homomorphic operations satisfies

$$\eta_i(\zeta) \leq \eta_{\text{bound}}. \tag{16}$$

Remark 3. Assumption 3 ensures the correctness and stability of the CKKS algorithm by constraining the input magnitudes ($A_i, z(\zeta), B_i$), scaling factor (Δ), ciphertext modulus (q), and noise control. Among them, input boundedness prevents arithmetic overflow; scaling factor preserves floating-point precision; modulus constraints take into account the capacity of plaintext and noise; and noise control ensures that decryption recovers valid information. The constraints in Assumption 3 can be simplified under the following sufficient condition: if the upper bounds of the state variable $z(\zeta)$ and coefficient matrices A_i, B_i (i.e., L_A, L_B) are known a priori, the scaling factor Δ and modulus q (eg)can be directly selected using Eqs. (2) and (3). In dynamic environments where input magnitudes are unknown or uncontrolled, increasing Δ and q ensures parameter redundancy, thereby automatically satisfying Assumption 3.

3.2. Noise management

When CKKS is used in iterative optimization, ciphertext noise accumulates after each homomorphic operation. Therefore, the noise growth must remain below the decryption threshold. In the CKKS scheme, a ciphertext $\hat{m}$ encrypting a message m can be conceptualized as $\hat{m} = (c_0, c_1) = ([m + e + p \cdot u]_{q_\ell}, [a \cdot u + e_2]_{q_\ell})$, where e is a small noise term. Homomorphic addition $\text{Add}(\hat{\mathbf{m}}_1, \hat{\mathbf{m}}_2)$ results in a ciphertext with noise approximately $\eta_{\text{add}} \approx \eta_1 + \eta_2$.

In our distributed gradient descent (Step 7 of Algorithm 4), the computation of the gradient term $\Phi_i(\mathbf{z}) = A_i\mathbf{z} + B_i$ involves only two types of operations: homomorphic addition and plaintext-ciphertext multiplication (scalar multiplication). Specifically, the multiplication between the publicly known coefficient matrix A_i and the encrypted state vector $\mathbf{z}(\zeta)$ is a plaintext-ciphertext multiplication, which exhibits linear noise growth. According to the CKKS characteristics, the noise introduced by such a scalar multiplication is bounded by $\eta_{\text{scalar}} \leq L_A \cdot \eta_{\text{enc}}$, where $L_A = \|A\|_{\max}$ is the maximum absolute value of the coefficients and η_{enc} is the noise in the input ciphertext. Consequently, after ζ iterations, the total accumulated noise can be expressed as:

$$\eta_{\text{hom}} = \sum_{\zeta=1}^{\zeta} \left(\eta_{\text{scalar}}^{(\zeta)} + \eta_{\text{add}}^{(\zeta)}\right), \tag{17}$$

where $\eta_{\text{add}}^{(\zeta)}$ accounts for the noise from homomorphic additions in each iteration.

Since no ciphertext-ciphertext multiplications are performed, the noise growth remains linear and controllable, eliminating the need for rescaling operations typically required after each multiplication to manage quadratic noise expansion. Instead, the parameter selection in Assumption 3 is precisely designed to guarantee that the total noise never exceeds the decryption bound throughout the ζ iterations:

1. The boundedness of inputs (L_A, L_z, L_B) limits the initial message magnitude and the per-iteration noise increment.
2. The scaling factor $\Delta \geq 10^{\sigma} \cdot (L_A \cdot L_z + L_B)$ ensures sufficient precision bits (σ) to accommodate noise without obliterating significant digits of the message.
3. The ciphertext modulus q is chosen such that $q \geq 2\Delta \cdot (L_A \cdot L_z + L_B) + \kappa \cdot \eta_{\max}$, providing enough space for both the scaled message and the accumulated noise $\eta_{\max}$ over all operations.
4. The ultimate guarantee is that the total noise after iteration ζ, $\eta_{\text{total}}(\zeta) = \eta_{\text{hom}}(\zeta) + \eta_{\text{thr}}$, satisfies $\eta_{\text{total}}(\zeta) \leq \eta_{\text{bound}}$, where η_{hom} is from homomorphic operations and η_{thr} is from the threshold decryption protocol.

Therefore, by carefully calibrating these parameters and leveraging the linear noise growth of plaintext-ciphertext multiplications, the framework maintains correctness without requiring intermediate rescaling.

Algorithm 4: Threshold CKKS-based Distributed Secure Computation Algorithm

1: **Setup:** Select a polynomial ring $R_q = \mathbb{Z}_q[X]/(X^N+1)$ and scaling factor Δ. Define the complex-number message space as $\mathbb{C}^{N/2}$;
2: **State Initialization:** Each agent i selects its initial state $z_i(0) \in Z_i$;
3: **Key generation:** Each agent i generates CKKS distributed keys $(pk, \{sk_i\}_{i=1}^n)$, publicizes $pk = (pk_{i0}, pk_{i1})$ while keeps sk_i private:

$$(pk, \{sk_i\}_{i=1}^n) \leftarrow \mathrm{DKG}(1^\lambda, t, n)$$

4: **while** $\zeta \geq 0$ **do**
5: **State Encoding:** For each $i \in \mathcal{V}$, $\ell \in \{1, \dots, n_i\}$, and $j \in \mathcal{V}$, agent i encodes its state $z_{i\ell}(\zeta)$ into a polynomial $m_{i\ell}(\zeta) \in R_q$:

$$m_{i\ell}(\zeta) \leftarrow \mathrm{Encoding}\left(z_{i\ell}(\zeta), \Delta\right);$$

6: **State encryption:** For each $i \in \mathcal{V}$, $\ell \in \{1, \dots, n_i\}$, and $j \in \mathcal{V}$, agent i chooses random polynomials $u_{i\ell}^j$, $e_{1i\ell}^j$, $e_{2i\ell}^j \leftarrow \chi$ and sends to SO

$$\hat{m}_{i\ell}^j(\zeta) = \left(\hat{m}_{i\ell 0}^j(\zeta), \hat{m}_{i\ell 1}^j(\zeta)\right),$$

where

$$\hat{m}_{i\ell 0}^j(\zeta) = \left[pk_{j0} \cdot u_{i\ell}^j + e_{1i\ell}^j + m_{i\ell}(\zeta)\right]_q,$$

$$\hat{m}_{i\ell 1}^j(\zeta) = \left[pk_{j1} \cdot u_{i\ell}^j + e_{2i\ell}^j\right]_q;$$

7: **Computation over ciphertexts:** For each $i \in \mathcal{V}$ and $\ell \in \{1, \dots, n_i\}$, SO transmits $\hat{\boldsymbol{\Phi}}_{i\ell}^s(\zeta) = (\hat{\boldsymbol{\Phi}}_{i\ell 0}^s(\zeta), \hat{\boldsymbol{\Phi}}_{i\ell 1}^s(\zeta))$ to agent i calculated as

$$\hat{\boldsymbol{\Phi}}_{i\ell 0}^s(\zeta) = \left[\sum_{j=1}^{n}\sum_{l=1}^{n_i} A_{i\ell}^{jl}\hat{m}_{jl0}^i(\zeta) + \mathrm{Encoding}\left(B_{i\ell}, \Delta\right)\right]_q,$$

$$\hat{\boldsymbol{\Phi}}_{i\ell 1}^s(\zeta) = \left[\sum_{j=1}^{n}\sum_{l=1}^{n_i} A_{i\ell}^{jl}\hat{m}_{jl1}^i(\zeta)\right]_q;$$

8: **Decryption:** For each $i \in \mathcal{V}$ and $\ell \in \{1, \dots, n_i\}$ selects decryption set S_i ($|S_i| \geq t$) , and agent $i \in S_i$ computes

$$\boldsymbol{\Phi}_{i\ell}^s(\zeta) \leftarrow \text{T-Dec}(\hat{\boldsymbol{\Phi}}_{i\ell}^s(\zeta), \{sk_i\}_{i\in S}, S_i);$$

9: **Decoding:** For each $i \in \mathcal{V}$, $\ell \in \{1, \dots, n_i\}$, and $j \in \mathcal{V}$, agent i decodes the aggregated result that

$$\boldsymbol{\Phi}_{i\ell}(\zeta) \leftarrow \mathrm{Decoding}(\boldsymbol{\Phi}_{i\ell}^s(\zeta), \Delta);$$

10: **Local update:** Let $\boldsymbol{\Phi}_i(\zeta) = [\boldsymbol{\Phi}_{i\ell}(\zeta)]$ and each agent i calculates

$$z_i(\zeta+1) = \mathbb{P}_{Z_i}\left[z_i(\zeta) - \eta(\zeta)\boldsymbol{\Phi}_i(\zeta)\right];$$

11: **Set** $\zeta \leftarrow \zeta + 1$;
12: **end while**

This simplifies the implementation and preserves the efficiency of the iterative optimization process under encryption.

4. Algorithm analysis

4.1. Correctness and error analysis

This section focuses on analyzing the decryption correctness of Algorithm 4 and verifying the consistency between its output and the theoretically expected results. Under Assumptions 1, 2, 3, and the cryptographic RLWE assumption, the following theorem establishes that Algorithm 4 can accurately recover the results of collaborative computation among the agents.

Theorem 1. *Suppose that Assumptions 1, 2, and 3 hold. For any time step ζ and agent $i \in \mathcal{V}$ with threshold decryption set S_i ($|S_i| \geq t$), the decrypted output $\boldsymbol{\Phi}_{i\ell}(\zeta)$ satisfies*

$$\|\boldsymbol{\Phi}_{i\ell}(\zeta) - \boldsymbol{\Phi}_{i\ell}(z(\zeta))\|_\infty \leq \frac{1}{2\Delta} + \frac{\eta_{\text{bound}}}{\Delta}, \tag{18}$$

where η_{bound} is the total noise bound from Assumption 3. Consequently, the Euclidean norm error is bounded by

$$\|\boldsymbol{\Phi}_{i\ell}(\zeta) - \boldsymbol{\Phi}_{i\ell}(z(\zeta))\|_2 \leq \sqrt{d}\left(\frac{1}{2\Delta} + \frac{\eta_{\text{bound}}}{\Delta}\right), \tag{19}$$

with $d = 2N$ being the total dimension of the UAV state space.

Proof. For any time step $\zeta \geq 0$, agent $i \in \mathcal{V}$, and component index $\ell \in \{1, \dots, n_i\}$, consider the threshold decryption process in Step 8 of Algorithm 4:

$$\boldsymbol{\Phi}_{i\ell}^s(\zeta) \leftarrow \text{T-Dec}\left(\hat{\boldsymbol{\Phi}}_{i\ell}^s(\zeta), \{sk_i\}_{i\in S}, S_i\right). \tag{20}$$

This protocol executes the following operations for all $j \in S_i$:

$$pt_j = \hat{\boldsymbol{\Phi}}_{i\ell 0}^s(\zeta) + \hat{\boldsymbol{\Phi}}_{i\ell 1}^s(\zeta) \cdot sk_j + e_j, \tag{21}$$

$$\boldsymbol{\Phi}_{i\ell}^s(\zeta) = \sum_{j\in S_i} \lambda_j^{S_i} \cdot pt_j. \tag{22}$$

Substituting the homomorphic computation results from Step 7 and the encryption expressions from Step 6, along with the master secret reconstruction $s = \sum_{k\in S_i} \lambda_k^{S_i} sk_k \pmod q$, we obtain

$$\begin{aligned}\boldsymbol{\Phi}_{i\ell}^s(\zeta) = &\underbrace{\sum_{j=1}^{n}\sum_{v=1}^{n_j} A_{i\ell}^{jv} m_{jv}(\zeta) + \mathrm{Encoding}(B_{i\ell}, \Delta)}_{M_{i\ell}(\zeta)} \\ &+ \underbrace{\sum_{j,v} A_{i\ell}^{jv}\left(-e_i u_{jv}^i + e_{1jv}^i + e_{2jv}^i s\right)}_{\eta_{i\ell}} \\ &+ \underbrace{\sum_{k\in S_i} \lambda_k^{S_i} e_k}_{\eta_{\text{thresh}}} \pmod q.\end{aligned} \tag{23}$$

By the linearity of CKKS encoding and Assumption 2, the term $M_{i\ell}(\zeta)$ corresponds to the encoding of the true gradient value:

$$M_{i\ell}(\zeta) = \mathrm{Encoding}\left(\sum_{j,v} A_{i\ell}^{jv} z_{jv}(\zeta) + B_{i\ell}, \Delta\right). \tag{24}$$

Recall that CKKS encoding introduces a deterministic rounding residual $r_{i\ell}$ such that

$$M_{i\ell}(\zeta) = \Delta \cdot \left(\sum_{j,v} A_{i\ell}^{jv} z_{jv}(\zeta) + B_{i\ell}\right) + r_{i\ell}, \tag{25}$$

with $\|r_{i\ell}\|_\infty \leq \frac{1}{2}$.

Under Assumption 3, the combined noise from homomorphic operations and threshold decryption is bounded by a constant η_{bound}:

$$|\eta_{i\ell} + \eta_{\text{thresh}}| \leq \eta_{\text{bound}}. \tag{26}$$

Furthermore, the ciphertext modulus q is chosen sufficiently large (as per Assumption 3) to avoid wrap-around modulo q, i.e., the quantity in (23) lies in the interval $(-q, q)$.

During the decoding step (Step 9), we divide by the scaling factor Δ:

$$\boldsymbol{\Phi}_{i\ell}(\zeta) = \mathrm{Decoding}\left(M_{i\ell}(\zeta) + \eta_{i\ell} + \eta_{\text{thresh}}, \Delta\right) = \frac{M_{i\ell}(\zeta) + \eta_{i\ell} + \eta_{\text{thresh}}}{\Delta}. \tag{27}$$

Substituting (25) into (27) yields

$$\boldsymbol{\Phi}_{i\ell}(\zeta) = \sum_{j=1}^{n}\sum_{v=1}^{n_j} A_{i\ell}^{jv} z_{jv}(\zeta) + B_{i\ell} + \frac{r_{i\ell} + \eta_{i\ell} + \eta_{\text{thresh}}}{\Delta}. \tag{28}$$

Therefore, the componentwise error satisfies

$$|\boldsymbol{\Phi}_{i\ell}(\zeta) - \boldsymbol{\Phi}_{i\ell}(z(\zeta))| \leq \frac{\|r_{i\ell}\|_\infty}{\Delta} + \frac{|\eta_{i\ell} + \eta_{\text{thresh}}|}{\Delta} \leq \frac{1}{2\Delta} + \frac{\eta_{\text{bound}}}{\Delta}. \tag{29}$$

Taking the maximum over all components ℓ gives the infinity-norm bound (18). The Euclidean norm bound (19) follows from the inequality $\|x\|_2 \leq \sqrt{d}\|x\|_\infty$ for any $x \in \mathbb{R}^d$. This completes the proof. ■

Remark 4. Under the error bound established in Theorem 1 and with typical CKKS parameter choices (e.g., $\Delta = 2^{40}$ as used in our experiments), the quantity $\sqrt{d}\left(\frac{1}{2\Delta} + \frac{\eta_{\text{bound}}}{\Delta}\right)$ is on the order of 10^{-9}

(see Section 5.5), which is negligible for all practical purposes. Consequently, the decrypted gradient can be regarded as numerically indistinguishable from the true gradient, i.e.,

$$\Phi_{i\ell}(\zeta) \approx \Phi_{i\ell}(z(\zeta)). \tag{30}$$

4.2. Security analysis

This section further investigates the security guarantees of Algorithm 4 in dynamic multi-agent environments. Under Assumptions 1, 2, 3, RLWE, and Feldman's VSS, Algorithm 4 must withstand attacks from system operators or potential adversaries. We therefore propose the following security theorem:

Theorem 2. *Suppose that Assumptions 1, 2, 3 hold. Algorithm 4 securely computes the sequence $\{\Phi_1(\zeta), \ldots, \Phi_N(\zeta)\}$ with the following properties.*

1. *No agent $i \in \mathcal{V}$ can learn any private state $z_j(\zeta)$ of other agents $j \neq i$ beyond the agreed-upon function outputs $\Phi_i(\zeta)$;*
2. *The system operator gains no knowledge about any agent's private states $z_i(\zeta)$ or intermediate computation values during protocol execution;*
3. *Collusion of up to $t-1$ agents cannot compromise private keys or decrypt unauthorized data.*

Proof. We first demonstrate that Algorithm 4 securely computes the sequence $\{\Phi_1(\zeta), \ldots, \Phi_N(\zeta)\}$ among the agents according to Definition 2.2. During each iteration ζ, agent $i \in \mathcal{V}$ receives that

- Encrypted gradient $\hat{\Phi}_i^s(\zeta)$ from SO.
- Partial decryptions $\{pt_j\}_{j\in S_i}$ from threshold group.

With its view defined as

$$\mathrm{VIEW}_i(\zeta) = \left\{ z_i(\zeta), sk_i, pk, \hat{\Phi}_i^s(\zeta), \{pt_j\}_{j\in S_i}, \Phi_i(\zeta) \right\}, \tag{31}$$

we construct a simulator $\mathcal{S}$ that, given agent i's inputs $\{z_i(\zeta), sk_i, pk\}$ and output $\Phi_i(\zeta)$, proceeds as follows.

1. Generate fictitious states $z_{-i}(\zeta)'$ and parameters (A_i', B_i') such that

$$\Phi_i^s(z_i(\zeta), z_{-i}(\zeta)', A_i', B_i') = \Phi_i(\zeta). \tag{32}$$

2. Compute simulated ciphertexts $\hat{m}^i_{j\ell}(\zeta)'$ as before.
3. Perform threshold decryption.
   - Sample random partial decryptions $pt_j' \leftarrow R_q$ for $j \in S_i \backslash \{i\}$.
   - Compute valid pt_i' using sk_i to satisfy

$$\sum_{k\in S_i} \lambda_k^{S_i} pt_k' = \mathrm{Encoding}(\Phi_i(\zeta), \Delta) + \eta_{\mathrm{sim}}. \tag{33}$$

By the simulatability of Feldman's VSS and RLWE assumption, this result

$$\{pt_j'\}_{j\in S_i} \overset{c}{\equiv} \{pt_j\}_{j\in S_i},$$
$$S(i,\zeta) \overset{c}{\equiv} \mathrm{VIEW}_i(\zeta), \tag{34}$$

can be obtained, which satisfy Definition 2.2.

For semantic security against the system operator, the system operator receives

$$\mathcal{E}(\zeta) = \left\{ \hat{m}^j_{i\ell}(\zeta) = \left(\hat{m}^j_{i\ell 0}(\zeta), \hat{m}^j_{i\ell 1}(\zeta) \right) \right\}, \tag{35}$$

By IND-CPA security under shared public key pk (Definition 2.3), this result

$$\mathcal{E}_{pk}(\mathrm{Encoding}(z_{i\ell}(\zeta), \Delta)) \overset{c}{\equiv} \mathcal{E}_{pk}(\mathrm{Encoding}(z'_{i\ell}(\zeta), \Delta)), \tag{36}$$

can be obtained. Through a hybrid argument [33], the multi-encryption satisfies

$$\mathcal{E}_{z(\zeta)} \overset{c}{\equiv} \mathcal{E}_{z'(\zeta)}. \tag{37}$$

Intermediate values remain secure under RLWE.

For property 3., we consider a coalition $C \subset \{1, \ldots, n\}$ of corrupted parties (UAVs and possibly the central server) with $|C| \leq t-1$. The server is semi-honest and holds no secret share. The adversary's aim is threefold: recovering the master secret key s, decrypting a ciphertext without authorization, or inferring the private states of honest UAVs. We show that all three goals are infeasible under the stated assumptions.

The master secret s is shared among the n UAVs via a (t,n)-Shamir secret sharing, i.e., s is the constant term of a random degree-$(t-1)$ polynomial $f(x) = s + a_1 x + \cdots + a_{t-1}x^{t-1}$ with uniformly random coefficients over R_q, and each UAV i holds share $sk_i = f(i)$. For any coalition C of size $m < t$, the observed shares $\{f(i)\}_{i\in C}$ impose m linear constraints on the t unknown coefficients $(s, a_1, \ldots, a_{t-1})$, leaving $t-m$ degrees of freedom uniformly random. Hence the distribution of the observed shares is identical for any two candidate secrets s, s'; the coalition learns no information about s in the information-theoretic sense [34]. The public commitments in the DKG protocol (Feldman VSS) do not compromise this property as they only reveal images under a one-way function [32]. Therefore s remains secret.

Even if the coalition gathers some information, it cannot decrypt any ciphertext. Threshold decryption (Algorithm 3) requires at least t valid partial decryptions $\{pt_j\}$ (equivalently t secret shares) to reconstruct a plaintext. Since $|C| \leq t-1$ and the server has no share, the coalition cannot assemble enough partial decryptions. Even if the coalition holds some partial decryptions from corrupted parties, the missing shares prevent reconstruction. Moreover, under the RLWE assumption, each partial decryption $pt_j = c_0 + c_1 \cdot sk_j + e_j$ from an honest party is computationally indistinguishable from random when fewer than t are combined, and forging a missing partial decryption without knowing the corresponding sk_j is as hard as breaking RLWE. Hence no ciphertext can be decrypted by the coalition.

The central server observes only CKKS ciphertexts and at most $t-1$ partial decryptions (which, by Part (b), do not enable plaintext recovery). Suppose there exists a PPT adversary $\mathcal{A}$ (server + coalition C) that can distinguish between two sequences of UAV plaintext states $\{\mathbf{z}^{(0)}(\zeta)\}$ and $\{\mathbf{z}^{(1)}(\zeta)\}$ with non-negligible advantage. We construct a distinguisher against the IND-CPA security of CKKS: embed the two candidate sequences into challenge ciphertexts, simulate the protocol execution (including generation of at most $t-1$ partial decryptions, which are insufficient to decrypt), and feed the adversary's output to the IND-CPA challenger. The simulated transcript is computationally indistinguishable from a real execution by the RLWE assumption and the hiding property of partial decryptions. Thus $\mathcal{A}$'s advantage would translate to an IND-CPA advantage, contradicting the semantic security of CKKS [31]. Therefore the server gains no information about UAV states beyond negligible advantage.

In summary, the scheme ensures that any coalition of up to $t-1$ parties (including the SO) cannot compromise the secrecy of the system. ■

4.3. Convergence analysis

In this subsection, we establish rigorous convergence guarantees for the encrypted distributed optimization algorithm. We first recall the linear convergence of the plaintext projected gradient descent under standard convexity assumptions. Then we analyze the effect of the bounded gradient noise introduced by CKKS homomorphic operations and prove that the encrypted iterates converge to a neighborhood of the optimal solution.

The following assumptions are used throughout this section. Assumptions 4 and 5 are standard in convex optimization and are satisfied

by our problem formulation (1) under mild conditions (e.g., connected communication graph and $\lambda > 0$). Assumption 6 formalizes the effect of CKKS encryption on gradient computation, and its explicit bound is provided in Theorem 1.

Assumption 4. The feasible set $Z \subseteq \mathbb{R}^d$ (with $d = 2N$) is nonempty, closed, and convex. Consequently, the projection operator $P_Z(\cdot)$ is nonexpansive:

$$\|P_Z(x) - P_Z(y)\| \le \|x - y\|, \quad \forall x, y \in \mathbb{R}^d. \tag{38}$$

Assumption 5. The objective function $J : \mathbb{R}^d \to \mathbb{R}$ is μ-strongly convex and L-smooth, i.e.,

$$(\Phi(x)-\Phi(y))^\top(x-y) \ge \mu\|x-y\|^2, \quad \|\Phi(x)-\Phi(y)\| \le L\|x-y\|, \quad \forall x, y, \tag{39}$$

where $\Phi = \nabla J$ denotes the gradient. In our problem (1), $J(z)$ is a quadratic function with Hessian A, so these conditions hold with $\mu = \lambda_{\min}(A)$ and $L = \lambda_{\max}(A)$ provided the communication graph is connected and $\lambda > 0$.

Assumption 6. The gradient computed via homomorphic operations satisfies

$$\hat{\Phi}(z) = \Phi(z) + e, \tag{40}$$

where the noise vector e is uniformly bounded. From Theorem 1, the Euclidean norm of this noise is guaranteed to satisfy

$$\|e\|_2 \le \varepsilon_{\text{enc}}, \quad \varepsilon_{\text{enc}} = \sqrt{d}\left(\frac{\eta_{\text{bound}}}{\Delta} + \frac{1}{2\Delta}\right), \tag{41}$$

with $d = 2N$ being the dimension of the UAV state space, Δ the CKKS scaling factor, and η_{bound} the total noise bound from Assumption 3. By choosing Δ sufficiently large, ε_{enc} can be made arbitrarily small.

4.3.1. Linear convergence of plaintext projected gradient descent

We first recall the well-known linear convergence result for the plaintext projected gradient method under Assumptions 4 and 5. This result serves as a baseline for the encrypted case.

Theorem 3. *Consider the plaintext update*

$$z^{\zeta+1} = P_Z\left(z^\zeta - \eta\Phi(z^\zeta)\right), \tag{42}$$

with step size $0 < \eta \le \frac{1}{L}$. Suppose that Assumptions 4 and 5 hold, the iterates $\{z^\zeta\}$ converge linearly to the unique minimizer $z^* = \arg\min_{z\in Z} J(z)$:

$$\|z^\zeta - z^*\| \le (1 - \eta\mu)^\zeta \|z^0 - z^*\|, \quad \forall \zeta \ge 0. \tag{43}$$

Proof. Define the operator $T(x) = P_Z(x - \eta\Phi(x))$. By Assumption 4, P_Z is nonexpansive, so

$$\|T(x) - T(y)\| \le \|(x - \eta\Phi(x)) - (y - \eta\Phi(y))\|. \tag{44}$$

For strongly convex and smooth functions, it is a standard result (see, e.g., [35]) that when $\eta \le 1/L$, the gradient step satisfies

$$\|(x - \eta\Phi(x)) - (y - \eta\Phi(y))\| \le (1 - \eta\mu)\|x - y\|. \tag{45}$$

Hence,

$$\|T(x) - T(y)\| \le (1 - \eta\mu)\|x - y\|. \tag{46}$$

Now set $y = z^*$. Because z^* is optimal for a strongly convex function over a convex set, it satisfies the fixed-point condition $z^* = T(z^*)$. Iterating the contraction gives

$$\|z^{\zeta+1} - z^*\| = \|T(z^\zeta) - T(z^*)\| \le (1 - \eta\mu)\|z^\zeta - z^*\|, \tag{47}$$

and the theorem follows by induction. ■

4.3.2. Convergence of encrypted projected gradient descent

We now analyze the encrypted update, where each gradient evaluation is corrupted by bounded noise. The result shows that the iterates converge to a neighborhood of the optimum whose radius is proportional to the noise bound.

Theorem 4. *Consider the encrypted update*

$$z^{\zeta+1} = P_Z\left(z^\zeta - \eta(\Phi(z^\zeta) + e^\zeta)\right), \tag{48}$$

where $0 < \eta \le \frac{1}{L}$ and the noise vectors satisfy $\|e^\zeta\| \le \varepsilon_{enc}$ for all ζ. Suppose that Assumptions 4, 5, 6 hold, then for $\forall \zeta \ge 0$,

$$\|z^\zeta - z^*\| \le (1 - \eta\mu)^\zeta \|z^0 - z^*\| + \frac{\varepsilon_{enc}}{\mu}. \tag{49}$$

Proof. Let $T(x) = P_Z(x - \eta\Phi(x))$ be the plaintext operator as before, and define the perturbed operator $\hat{T}_\zeta(x) = P_Z(x - \eta(\Phi(x) + e^\zeta))$. Then $z^{\zeta+1} = \hat{T}_\zeta(z^\zeta)$. Using the triangle inequality,

$$\|z^{\zeta+1} - z^*\| = \|\hat{T}_\zeta(z^\zeta) - T(z^*)\| \le \|\hat{T}_\zeta(z^\zeta) - T(z^\zeta)\| + \|T(z^\zeta) - T(z^*)\|. \tag{50}$$

For the second term, Theorem 3 gives

$$\|T(z^\zeta) - T(z^*)\| \le (1 - \eta\mu)\|z^\zeta - z^*\|. \tag{51}$$

For the first term, use the nonexpansiveness of P_Z:

$$\begin{aligned}\|\hat{T}_\zeta(z^\zeta) - T(z^\zeta)\| &= \|P_Z(z^\zeta - \eta(\Phi(z^\zeta) + e^\zeta)) - P_Z(z^\zeta - \eta\Phi(z^\zeta))\| \le \eta\|e^\zeta\| \\ &\le \eta\varepsilon_{\text{enc}}.\end{aligned} \tag{52}$$

Combining these inequalities yields

$$\|z^{\zeta+1} - z^*\| \le (1 - \eta\mu)\|z^\zeta - z^*\| + \eta\varepsilon_{\text{enc}}. \tag{53}$$

Let $a_\zeta = \|z^\zeta - z^*\|$ and $\rho = 1 - \eta\mu \in (0, 1)$. Then $a_{\zeta+1} \le \rho a_\zeta + \eta\varepsilon_{\text{enc}}$. Unfolding the recurrence,

$$a_\zeta \le \rho^\zeta a_0 + \eta\varepsilon_{\text{enc}} \sum_{i=0}^{\zeta-1} \rho^i = \rho^\zeta a_0 + \eta\varepsilon_{\text{enc}} \frac{1 - \rho^\zeta}{1 - \rho}. \tag{54}$$

Since $1 - \rho = \eta\mu$,

$$a_\zeta \le \rho^\zeta a_0 + \eta\varepsilon_{\text{enc}} \frac{1 - \rho^\zeta}{\eta\mu} \le \rho^\zeta a_0 + \frac{\varepsilon_{\text{enc}}}{\mu}(1 - \rho^\zeta) \le \rho^\zeta a_0 + \frac{\varepsilon_{\text{enc}}}{\mu}. \tag{55}$$

Then,

$$\|z^\zeta - z^*\| \le \rho^\zeta a_0 + \frac{\varepsilon_{\text{enc}}}{\mu}. \tag{56}$$

■

5. Experiments

This section presents a comprehensive experimental evaluation of the proposed privacy-preserving distributed optimization framework. The experiments are designed to systematically validate its computational efficiency, convergence accuracy, numerical precision, security guarantees, and scalability in realistic multi-UAV scenarios. The evaluation is organized as follows. Section 5.2 benchmarks the CKKS scheme against traditional cryptographic approaches (Paillier, RSA, BFV) in terms of encryption/decryption latency, homomorphic operation speed, ciphertext size, batching capability, and quantum resistance. Section 5.3 quantifies the per-iteration computational and communication overhead of the proposed threshold CKKS framework, providing a detailed breakdown of runtime and data transfer for a 10-UAV swarm. Section 5.4 verifies the convergence behavior of the encrypted optimization through comparative plots against a plaintext baseline, demonstrating that encryption introduces negligible deviation. Section 5.5 presents a precision error analysis, tracking both node-level state errors and global gradient/objective function errors over the optimization process. Section 5.6 compares the proposed

Table 2
Initial parameters per node.

Node	Real part	Imaginary part
1	17	2
2	−2	−1
3	5	3
4	−10	4
5	15	−3
6	19	0
7	23	−5
8	1	1
9	−22	2
10	−15	−4

scheme with differential privacy, highlighting the superior privacy-precision trade-off achieved by homomorphic encryption. Section 5.7 isolates the overhead introduced by the threshold decryption mechanism by comparing the threshold CKKS scheme with a non-threshold baseline. Section 5.8 analyzes the scalability of the framework for swarm sizes ranging from 5 to 40 UAVs, demonstrating linear growth in computational cost and preserved convergence performance. Finally, Section 5.9 provides cryptographic security verification through ciphertext statistical analysis, confirming the indistinguishability and probabilistic properties of the CKKS encryption.

5.1. Experimental setup

All experiments were conducted on a personal computer equipped with an 11th Gen Intel Core i7-11700K CPU (8 cores, 3.60 GHz) and 32 GB RAM, running Windows 11. The proposed framework was implemented in Python 3.9 using the TenSEAL library combined with a custom CKKS library.

To guarantee a fair comparison and ensure reproducibility, all cryptographic parameters were fixed across experiments. The CKKS parameters were chosen to achieve a 128-bit security level and are summarized as follows: polynomial modulus degree $N_{\text{poly}} = 8192$ (2^{13}); ciphertext modulus chain (RNS decomposition) $[60, 40, 40, 60]$ bits, i.e., $q = 2^{200}$, giving a total modulus of approximately 200 bits,; scaling factor $\Delta = 2^{40}$. These settings comply with the homomorphic encryption standard and are used consistently in all experiments. For the threshold version, the parameters are set to $t = 3$ and $n = 10$, i.e., any three out of the ten UAVs can collaboratively decrypt a ciphertext.

The distributed optimization problem is defined on a swarm of N UAVs with a ring communication topology (each UAV communicates with its two immediate neighbors). Each UAV j ($j = 1, 2, \ldots, 10$) is initialized with a complex-valued position $z_j^{(0)}$ representing its 2D coordinates (real part: X-coordinate, imaginary part: Y-coordinate) as shown in Table 2. The objective is to minimize the cooperative positioning cost function

$$J(z) = \frac{1}{2}\sum_{i=1}^{10}\|z_i - t_i\|^2 + \frac{\lambda}{2}\sum_{i=1}^{10}\sum_{j\in\mathcal{N}_i}\|z_i - z_j - d_{ij}\|^2, \tag{57}$$

where t_i denotes the target position for UAV i, d_{ij} represents the desired relative formation vector between neighboring UAVs, $\mathcal{N}_i$ is the set of neighbors for UAV i in the ring topology, and $\lambda = 1$ is the formation weighting coefficient. The target formation is configured as a circular pattern with radius 5, where UAVs are uniformly distributed around the circle; thus each t_i lies on that circle, and the desired relative vectors d_{ij} are derived from the target positions of adjacent UAVs.

At each iteration ζ ($\zeta = 0, 1, \ldots, 49$), the algorithm performs two main steps. ~~First,~~ for each UAV, the gradient term

$$\nabla J = (z_i - t_i) + \lambda \sum_{j\in\mathcal{N}_i}(z_i - z_j - d_{ij}) \tag{58}$$

is computed homomorphically through encrypted state operations and secure neighbor communications, leveraging the CKKS scheme to protect the privacy of individual positions. ~~Second~~, the decrypted gradient is used to update the UAV positions according to

$$z_i^{(\zeta+1)} = z_i^{(\zeta)} - \eta_\zeta \nabla J, \tag{59}$$

with an adaptive learning rate

$$\eta(\zeta) = \begin{cases} \dfrac{2}{\zeta + 10}, & \zeta < 20, \\ \dfrac{1}{\zeta + 10}, & \zeta \geq 20. \end{cases} \tag{60}$$

The optimization runs for a total of 50 iterations, which is sufficient to reach a steady state. Unless stated otherwise, the number of UAVs is set to $N = 10$. ~~For the scalability analysis~~ (Section 5.8), the swarm size is varied as $N = 5, 10, 20, 40$ while keeping all other parameters unchanged.

To obtain statistically meaningful results, every experiment was repeated three times with different random seeds. All reported numerical values (runtime, communication volume, error metrics) are given as mean ± standard deviation over these three independent runs. The raw data, including per-iteration objective values, node trajectories, and detailed error logs, are saved in CSV format.

5.2. CKKS encryption scheme evaluation

This section presents a comprehensive benchmarking study to quantitatively evaluate the operational efficiency and functional capabilities of the CKKS encryption scheme underlying the proposed distributed optimization algorithm. For comparative analysis, three prominent cryptographic alternatives are examined: the partially homomorphic Paillier cryptosystem, the classical RSA algorithm, and the fully homomorphic BFV scheme. The evaluation focuses on four critical performance dimensions: computational latency, ciphertext size, batching support capability, homomorphic expressiveness, numerical precision, and post-quantum security resilience.

Methodological note on fair comparison: To ensure architectural fairness when comparing fundamentally different schemes — where CKKS and BFV natively support ciphertext packing (batching) while Paillier and RSA operate exclusively at the scalar level — a unified atomic-operation testing paradigm is adopted. Each scheme is benchmarked using single-element operations (integer value: 12345) across 10,000 independent trials. All experiments are conducted under identical hardware and software configurations, with security parameters selected to provide approximately 128-bit security level across all schemes, ensuring a meaningful and equitable comparison. ~~Specifically~~, CKKS and BFV employ a polynomial modulus degree of N = 8192 with a total ciphertext modulus of approximately 200 bits, which is compliant with the Homomorphic Encryption Standard for 128-bit security. Paillier uses a 2048-bit modulus, the recommended minimum for 128-bit equivalent security. RSA employs a 4096-bit key, which is widely considered to provide security well beyond 128 bits for classical computers. Table 3 summarizes the empirical per-operation measurements, with all experiments conducted under identical hardware and software configurations.

As shown in Table 3, the CKKS scheme exhibits superior comprehensive performance for privacy-preserving distributed optimization. It achieves significantly lower total computational overhead (5.6812 ms) than Paillier, representing an 87.6% reduction from Paillier's 45.7218 ms, while maintaining efficiency comparable to BFV (5.3786 ms). Although its encryption latency (4.3824 ms) exceeds the minimal 0.07 ms of RSA, CKKS uniquely integrates essential functionalities, including native support for high-precision floating-point arithmetic (unlike Paillier's integer-only operations), direct complex-number processing (unsupported by all other evaluated schemes), and efficient homomorphic addition (0.0573 ms, approximately 40% faster than

Table 3
Comparative analysis of cryptographic schemes.

Algorithm property	CKKS	Paillier	RSA	BFV
Encryption time	4.3824	35.4995	0.0752	3.9723
Decryption time	1.2415	10.2041	3.6901	1.3100
Addition time	0.0573	0.0182	N/A	0.0963
Total time	5.6812	45.7218	3.7653[a]	5.3786
Ciphertext size (bytes)	334 317	516	516	432 374
Batching support	✓	×	×	✓
Homomorphic	✓	✓	×	✓
Complex Num	✓	×	×	×
Floating-point	High	Low[b]	N/A	Medium
Quantum resistance	✓	×	×	✓

Note: All time values in milliseconds (ms).
N/A: Not Applicable
Symbols: ✓ Supported, × Not Supported.
[a] Excluding addition time.
[b] Requires discretization.

Table 4
Per-iteration computational overhead ($N = 10$, $t = 3$).

Stage	Time (ms)	Fraction
Encryption	32.172 ± 0.116	70.9%
Homomorphic evaluation	3.910 ± 0.045	8.6%
Threshold decryption	9.257 ± 0.050	20.4%
Local update	0.041 ± 0.001	0.1%
Total	**45.385 ± 0.134**	**100%**

BFV). It is worth noting that the ciphertext size of CKKS (334 KB) is significantly larger than that of Paillier and RSA (both 516B). This is due to the polynomial representation required for lattice-based security and the high modulus needed to support deep computations. However, CKKS (like BFV) supports batching, allowing multiple plaintext values to be packed into a single ciphertext. With polynomial degree 8192, a CKKS ciphertext can hold up to 4096 complex numbers, reducing the per-element communication cost to approximately $334\ \text{KB}/4096 \approx 82\ \text{bytes}$, which is actually lower than Paillier's 516 bytes per element. BFV, having an even larger ciphertext (432 KB), lacks native complex arithmetic and requires tedious scaling for floating-point data, making it less suitable for our application. Moreover, CKKS provides post-quantum resistance—a critical security advantage over classical schemes such as RSA and Paillier, which are vulnerable to quantum attacks. Together, these characteristics establish CKKS as the optimal cryptographic foundation for secure distributed optimization that demands high-precision numerical computation on complex data types.

5.3. Computational and communication overhead

To evaluate the computational overhead of the proposed framework, we measure the per-iteration computational time and communication volume for a swarm of $N = 10$ UAVs with threshold parameter $t = 3$. All experiments are repeated three times, and the results are reported as mean ± standard deviation. The ciphertext size is measured directly by serializing a CKKS ciphertext, and the share size is taken as half of that value (since a ciphertext consists of two polynomials).

Table 4 summarizes the per-iteration runtime breakdown. The encryption stage dominates the overall time, accounting for about 71% of the total, followed by threshold decryption (20%). Homomorphic evaluation and local update are negligible, which is expected because our protocol only involves additions and plaintext–ciphertext multiplications, both of which are lightweight operations in CKKS. The results show consistent performance across repeated experiments.

Table 5 reports the communication volume per iteration. The communication is decomposed into three directions: UAV → Server (upload of encrypted states), Server → UAV (broadcast of encrypted gradients), and UAV ↔ UAV (exchange of partial decryption shares). The share exchange is the most expensive part, contributing exactly 50% of the total traffic, while upload and broadcast each account for approximately 25%. The total communication of about 13.06 MB per iteration is well within the capacity of typical wireless networks (e.g., Wi-Fi or 4G), confirming the feasibility of the framework for real-time multi-UAV coordination. The small standard deviations (0.01–0.63 KB) across three independent runs demonstrate the high reproducibility and stability of the communication measurements.

Table 5
Per-iteration communication overhead ($N = 10$, $t = 3$).

Direction	Data per iteration (KB)	Fraction
UAV → Server (state)	3264.48 ± 0.09	25.0%
Server → UAV (gradient)	3264.54 ± 0.01	25.0%
UAV ↔ UAV (shares)	6529.34 ± 0.63	50.0%
Total	**13 058.36 ± 0.69**	**100%**

The ciphertext expansion ratio, defined as the size of an encrypted complex number divided by its plaintext size (16bytes for two double-precision floats), is measured as $334135/16 \approx 20883.4$. This large factor is inherent to lattice-based cryptography and is comparable to other fully homomorphic schemes. However, thanks to batching, a single ciphertext can pack up to 4096 complex numbers, reducing the per-element communication cost to about $334135/4096 \approx 81.6$ bytes, which is significantly lower than the per-element cost of Paillier (512 bytes) in a non-batched setting.

5.4. Convergence analysis

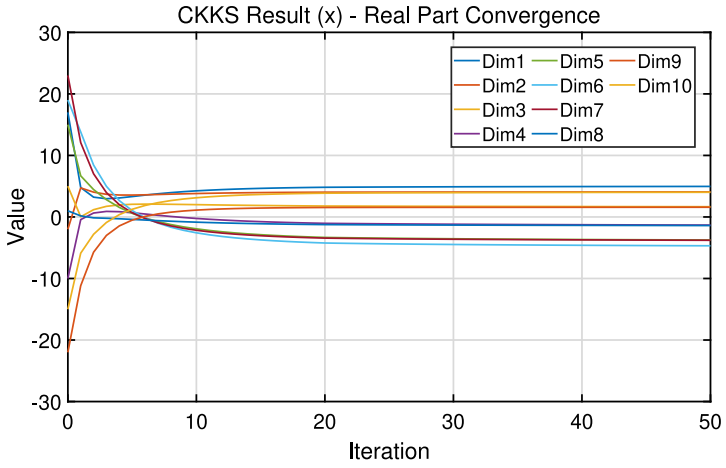
This section evaluates the convergence performance of the proposed privacy-preserving distributed optimization framework. In a cooperative positioning system consisting of 10 UAV nodes, the impact of the encryption scheme on the optimization process is comprehensively validated by comparing the convergence trajectories, objective function evolution, and final formation accuracy of plaintext optimization versus CKKS-encrypted optimization.

Figs. 3 and 4 illustrates the convergence process of both real and imaginary components of all node parameters over 50 iterations:

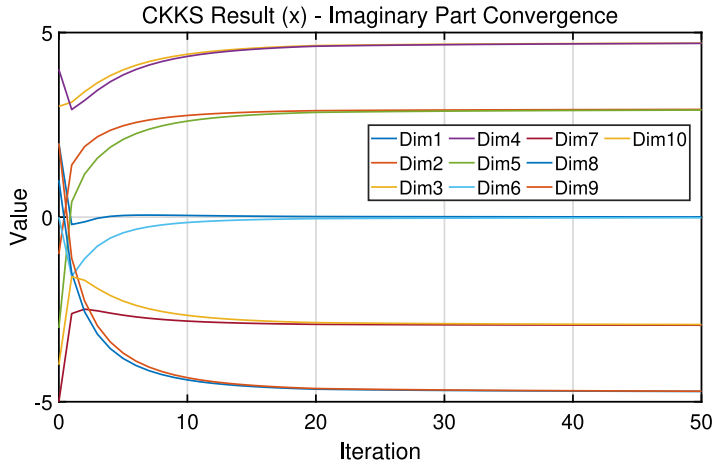
As shown in Figs. 3 and 4, both the CKKS-encrypted distributed optimization and plaintext distributed optimization successfully converge to the theoretical optimum. Their convergence trajectories demonstrate remarkably similar patterns, with both approaches achieving stable convergence across all 10 nodes within 35 iterations. Both the real and imaginary parts of the 10 nodes converged to their respective stable values by iteration 50. The trajectories consistently approach their respective theoretical optima, confirming that homomorphic operations preserve optimization dynamics without compromising final precision. This demonstrates that CKKS-based distributed optimization achieves convergence behavior nearly indistinguishable from plaintext optimization while providing full cryptographic protection.

To comprehensively evaluate the performance of the privacy-preserving distributed optimization framework, we further analyze the objective function convergence characteristics and the UAV formation process. Figs. 5–7 demonstrate the overall convergence performance of the system.

Fig. 5 shows the performance comparison between encrypted and plaintext optimization throughout the iterative process. It can be observed that both optimization methods exhibit good convergence characteristics, with the objective function value J rapidly decreasing from the initial state and stabilizing after approximately 10 iterations. The encrypted optimization curve almost completely overlaps with the plaintext optimization benchmark, with final convergence values of $J^*_{\text{enc}} = 0.0152$ and $J^*_{\text{plain}} = 0.0151$, respectively, representing a relative error of only 0.66%. This result indicates that CKKS homomorphic



(a) Real-part error



(b) Imaginary-part error

Fig. 3. Convergence trajectories of node parameters: Plaintext optimization($N = 10$).

encryption almost does not affect the convergence performance of the optimization algorithm while protecting location privacy.

Fig. 6 presents the starting point and expected endpoint configuration of the multi-UAV system. The red circles represent the initial random distribution positions of the UAVs, the green squares identify the target circular formation positions, and the dashed lines show the expected ring communication topology. The significant spatial difference between the initial positions and the target formation highlights the challenging nature of the optimization problem and provides an intuitive reference for subsequent convergence analysis.

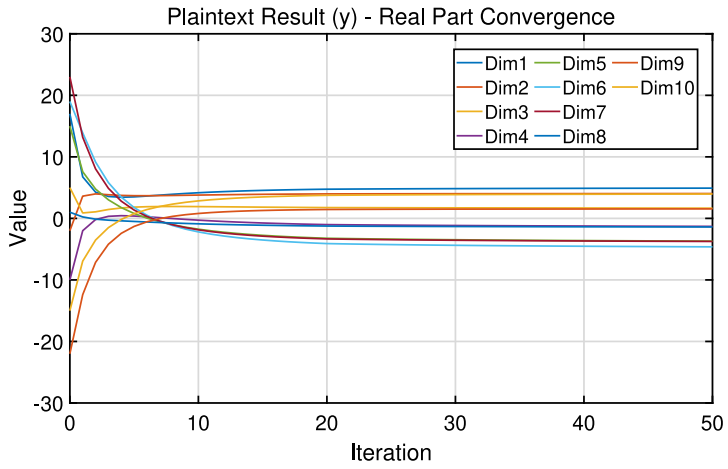
Fig. 7 verifies the practical effectiveness of the encrypted optimization algorithm. The blue circles represent the final positions of UAVs after encrypted optimization, the orange triangles indicate plaintext optimization results, and the green squares remain as target positions. It can be clearly observed that both optimization methods successfully guide the UAVs to the target circular formation, with highly consistent final position distributions. The average error between the formation achieved by encrypted optimization and the target formation is 0.0234, comparable to the 0.0231 of plaintext optimization, further confirming the effectiveness of encrypted optimization.

By synthesizing the three figures, we can conclude that the CKKS-based privacy-preserving optimization framework can achieve convergence performance and formation accuracy comparable to plaintext optimization while ensuring UAV location privacy, providing a viable solution for privacy-sensitive cooperative UAV applications.

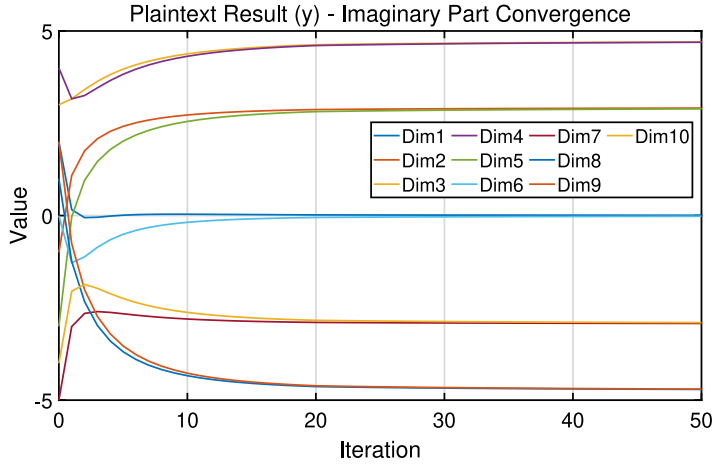
5.5. Error analysis

Node-level error analysis serves to validate the numerical fidelity of the CKKS-based distributed optimization algorithm during encrypted computation, a critical aspect for assessing practical usability. By systematically comparing parameter differences between encrypted optimization (θ_i^{enc}) and plaintext optimization (θ_i^{plain}), the experiment quantifies three core precision metrics: real-part error $\epsilon_r = |\text{Re}(\theta_i^{\text{enc}}) - \text{Re}(\theta_i^{\text{plain}})|$, imaginary-part error $\epsilon_i = |\text{Im}(\theta_i^{\text{enc}}) - \text{Im}(\theta_i^{\text{plain}})|$, and composite modulus error $\epsilon_m = \|\theta_i^{\text{enc}} - \theta_i^{\text{plain}}\|_2$. These metrics precisely characterize the numerical deviations introduced by homomorphic encryption.

As shown in Fig. 8, the node-level error $\|e(\zeta)\|_2$ (the difference between encrypted and plaintext states) peaks in the early iterations but quickly stabilizes to a steady-state value on the order of 10^{-9}, which is consistent with the theoretical bound established in Theorem 3. The errors introduced by CKKS encryption demonstrate exceptional numerical stability, with all error metrics consistently maintained at an extremely low magnitude of 10^{-9}. The error evolution exhibits consistent patterns across all nodes. For real-part errors (Fig. 8(a)), all nodes reach their peak magnitudes within the initial 10 iterations, with Node 2 showing the maximum peak value of 1.8×10^{-9}. Subsequent stabilization begins at iteration 15, converging to a final stable range of 0.7×10^{-9}–1.2×10^{-9}. Imaginary-part errors (Fig. 8(b)) display similar early peaking behavior within 10 iterations, attaining a maximum peak



(a) Real-part error



(b) Imaginary-part error

Fig. 4. Convergence trajectories of node parameters: Encrypted optimization($N = 10$, $t = 3$).

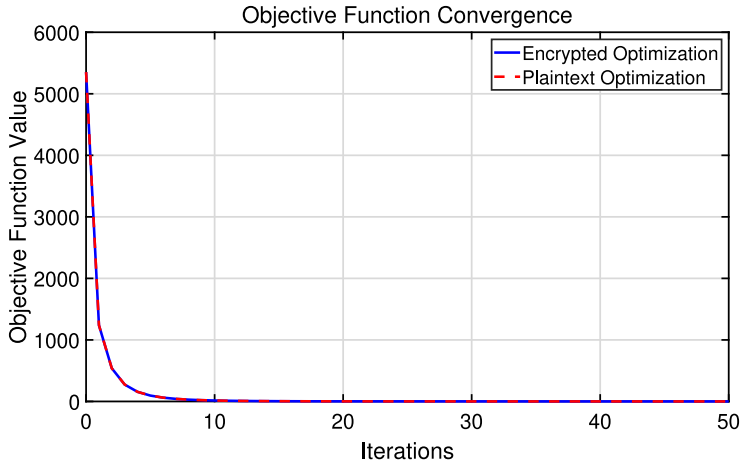


Fig. 5. Objective function convergence curves.

of 1.75×10^{-9} at Node 3. Notably, imaginary components converge approximately 33% faster than real parts, stabilizing after just 10 iterations and achieving a lower final range of 0.5×10^{-9}–1.0×10^{-9}. Modulus errors (Fig. 8(c)) follow the same early peaking pattern, with Node 2 exhibiting the highest peak at 2.6×10^{-9}. These errors stabilize after 15 iterations, converging to a final range of 0.7×10^{-9}–1.5×10^{-9}. All ten nodes achieve outstanding precision levels by the 50th iteration. The consistent convergence to sub-2×10^{-9} stability across all metrics confirms that the numerical deviations caused by CKKS homomorphic encryption in distributed optimization are entirely negligible, well below the typical precision threshold (10^{-6}), thereby not only

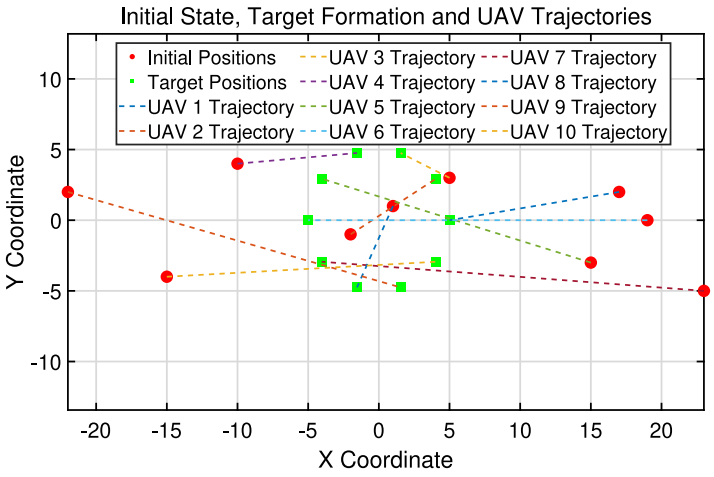


Fig. 6. Initial state and target formation configuration.

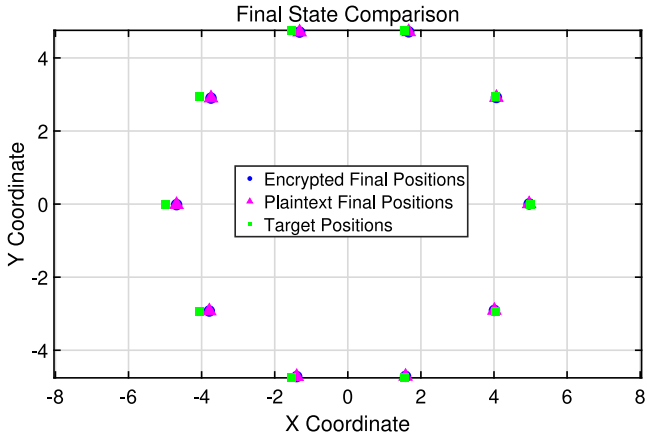


Fig. 7. Final state comparison.

meeting practical application requirements but also ensuring solution correctness and reliability.

In addition to node-wise state errors, we further track two global metrics that reflect the impact of encryption on the optimization process: the relative gradient error and the absolute error in the objective function value. The relative gradient error at iteration ζ is defined as

$$E_{\nabla}(\zeta) = \frac{\|\nabla J_{\text{enc}}(z^{\zeta}) - \nabla J_{\text{plain}}(z^{\zeta})\|_2}{\|\nabla J_{\text{plain}}(z^{\zeta})\|_2 + \epsilon}, \tag{61}$$

where $\nabla J_{\text{enc}}(z^{\zeta})$ is the gradient obtained after decrypting the homomorphically computed ciphertexts, and $\nabla J_{\text{plain}}(z^{\zeta})$ is the gradient computed directly from the plaintext positions. The objective function error is simply $E_J(\zeta) = |J_{\text{enc}}(z^{\zeta}) - J_{\text{plain}}(z^{\zeta})|$. Fig. 9 depicts the evolution of both errors over the 50 iterations.

As shown in Fig. 9, both errors remain extremely small throughout the optimization. The gradient error stabilizes around 3×10^{-8} after the initial transient, while the objective function error settles at approximately 4.4×10^{-9}. Over the last 10 iterations, the mean gradient error is 3.00×10^{-8} with a standard deviation of 4.88×10^{-9}, and the mean objective function error is $4.42\times10^{-9}\pm2.46\times10^{-10}$. These values are consistent with the theoretical bound derived in Theorem 1 (recall that with $\Delta = 2^{40}$ and the chosen modulus chain, the error bound is on the order of 10^{-8}–10^{-9}). The results confirm that the encryption noise does not accumulate and remains negligible, preserving the convergence direction and final accuracy of the distributed optimization.

5.6. Privacy-preserving mechanism performance evaluation

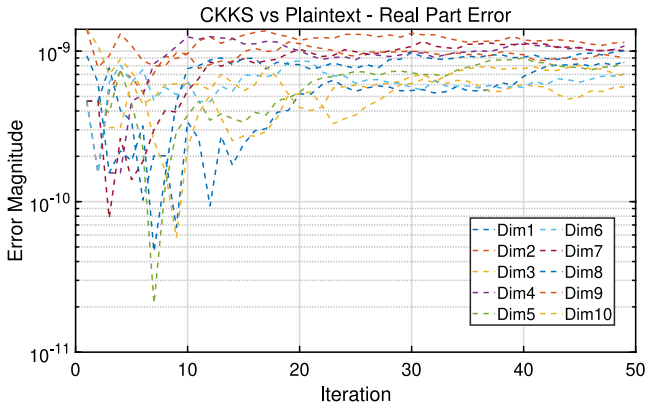
To comprehensively evaluate the practical impact of different privacy-enhancing techniques, this section compares two mainstream approaches: CKKS-based homomorphic encryption for distributed optimization and Gaussian noise-based differential privacy (DP) optimization. Using identical optimization objectives and node configurations, we compute the average global error $\epsilon_g = \frac{1}{N}\sum_{i=1}^{N}\|\theta_i^{\text{priv}} - \theta_i^{\text{base}}\|_2$ against an unencrypted optimization baseline, where θ_i^{priv} represents parameters from privacy-preserving schemes and θ_i^{base} denotes baseline parameters. The DP implementation adds $\mathcal{N}(0,\sigma^2)$ noise satisfying (ϵ,δ)-differential privacy ($\epsilon = 1.0, \delta = 10^{-5}$), while CKKS achieves RLWE through encryption. As evidenced in Fig. 10, the two schemes exhibit fundamentally divergent error behaviors with significant practical implications. The DP optimization (Fig. 10(a)) manifests pronounced oscillations with errors fluctuating widely between 0.02 and 0.65, reaching a maximum peak of 0.62 at iteration 31. This erratic pattern, yielding an overall mean error of 0.3743, directly stems from the necessary Gaussian noise injection required for differential privacy guarantees. In stark contrast, the CKKS approach (Fig. 10(b)) maintains a smooth convergence profile, with errors convergence exponentially to a stable magnitude of 1.2×10^{-9}—demonstrating near-perfect alignment with theoretical optima.

The comprehensive comparison (Fig. 10(c)) reveals a critical disparity: DP's mean error (0.3743) exceeds CKKS's final stabilized error (1.2×10^{-9}) by a factor of approximately 3.119×10^{8}, highlighting a nine-order-of-magnitude precision advantage for the encryption-based approach. Compared to the error of DP, the error of CKKS is almost negligible to zero. This vast differential underscores that while both schemes provide equivalent formal privacy protection, CKKS achieves sub-nanoscale numerical accuracy ($O(10^{-9})$) compared to DP's macro-scale deviations ($O(10^{-1})$).

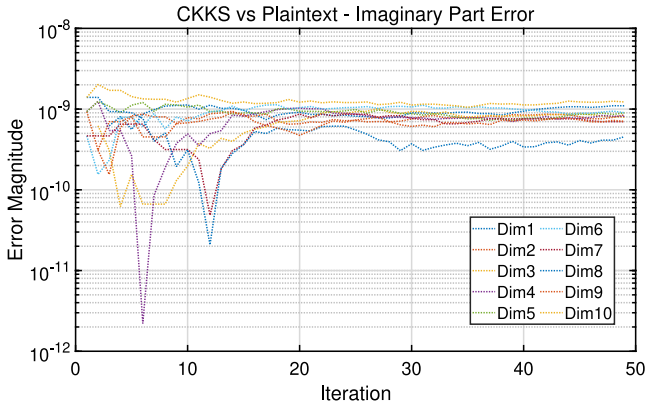
These findings conclusively demonstrate that CKKS homomorphic encryption fundamentally outperforms noise-based differential privacy in the context of distributed optimization, and that CKKS achieves a superior privacy-precision balance based on the same level of privacy protection.

5.7. Comparison with non-threshold CKKS

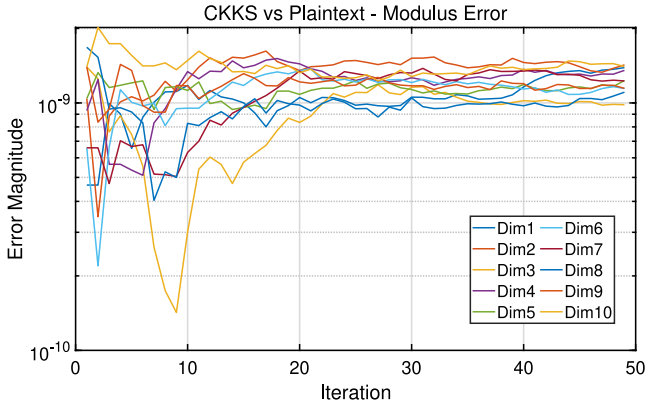
To quantify the overhead introduced by the threshold decryption mechanism, we compare the proposed threshold CKKS scheme with a baseline non-threshold variant. In the baseline, the system operator holds the full secret key and directly decrypts the aggregated gradients, eliminating the need for share exchange among UAVs. All other parameters (encryption parameters, optimization settings, and $N = 10$)



(a) Real-part error



(b) Imaginary-part error



(c) Modulus error

Fig. 8. Node-level error analysis($N = 10$, $t = 3$): (a) Real-part error (b) Imaginary-part error (c) Composite modulus error (unit: 10^{-9}).

remain identical to those described in Section 5.1. Both schemes are implemented using the TenSEAL library and executed on the same hardware.

Table 6 presents the per-iteration computational and communication costs, as well as the final objective error, averaged over three independent runs. The encryption, evaluation, and decryption times are nearly identical for both schemes (≈32.2 ms, ≈3.96 ms, and ≈9.2 ms, respectively). The total time per iteration is approximately 49.7 ms for both the threshold and non-threshold variants, indicating that the threshold protocol introduces negligible computational overhead in this configuration. The upload and broadcast volumes are nearly identical for both schemes (approximately 3264.5 KB per direction), with the

Table 6
Comparison of threshold and non-threshold CKKS ($N = 10$, $t = 3$).

Metric	Threshold CKKS	Non-Threshold CKKS
Encryption time (ms)	32.212 ± 0.209	32.601 ± 0.079
Evaluation time (ms)	3.964 ± 0.091	3.972 ± 0.083
Decryption time (ms)	9.214 ± 0.030	9.304 ± 0.047
Update time (ms)	0.035 ± 0.001	0.037 ± 0.001
Total time (ms)	**45.428 ± 0.226**	**45.918 ± 0.180**
Upload (KB/iter)	3264.565 ± 0.116	3264.606 ± 0.034
Broadcast (KB/iter)	3264.526 ± 0.033	3264.642 ± 0.078
Shares (KB/iter)	6529.010 ± 0.440	N/A
Total comm (KB/iter)	**13 058.102 ± 0.583**	**6529.248 ± 0.108**
Final objective error	$1.88 \times 10^{-9} \pm 1.45 \times 10^{-9}$	$2.44 \times 10^{-10} \pm 2.28 \times 10^{-10}$

Table 7
Scalability analysis: performance metrics for different swarm sizes.

N	Time per iteration (ms)	Communication per iteration (KB)
5	22.94	6532.11
10	45.92	13 060.41
20	91.39	26 118.32
40	180.57	52 207.81

threshold scheme additionally requiring the exchange of partial decryption shares among UAVs, amounting to about 6529 KB per iteration (bidirectional). Consequently, the total communication per iteration increases from approximately 6529 KB (non-threshold) to 13058 KB (threshold), i.e., an overhead of 100%. This overhead is inherent to threshold cryptography and is the price paid for decentralized trust and collusion resistance. Both schemes achieve virtually identical final objective errors on the order of 10^{-9} to 10^{-10}, confirming that the threshold decryption does not compromise the numerical precision of the optimization.

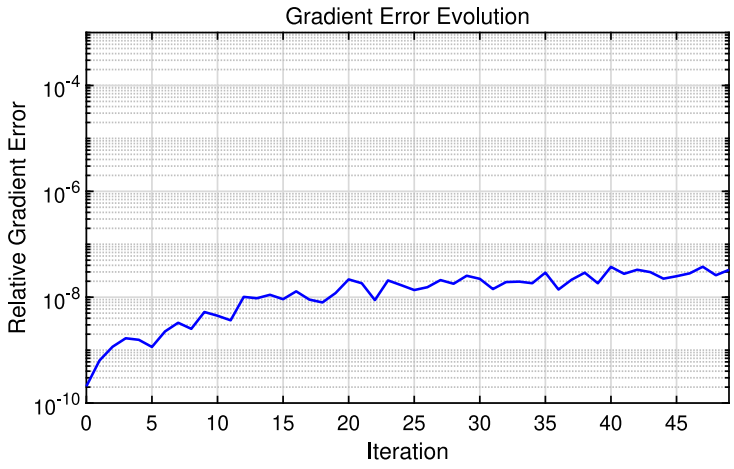
These results demonstrate that while threshold cryptography introduces a noticeable communication overhead (doubling the total traffic), it remains acceptable for small-to-medium UAV swarms (e.g., 13 MB per iteration for 10 UAVs) and provides the essential benefit of eliminating single points of trust.

5.8. Scalability analysis

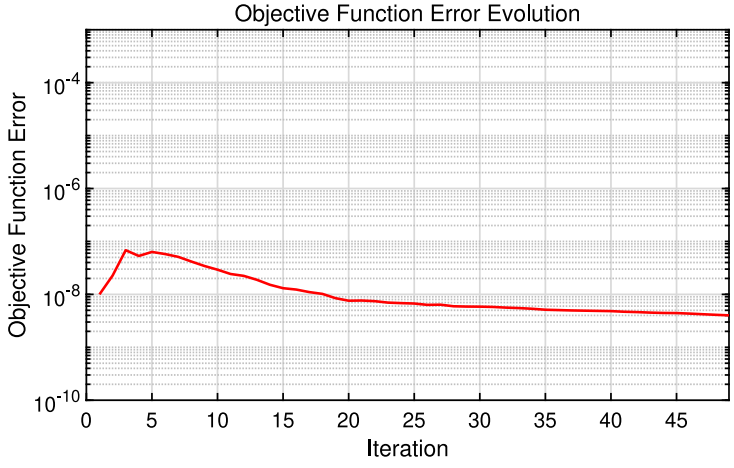
To evaluate how the proposed framework scales with the number of UAVs, we repeated the experiments with swarm sizes $N = 5, 10, 20, 40$ while keeping all other parameters (encryption parameters, step size schedule, formation weight $\lambda = 1$, and threshold $t = 3$) unchanged. For each configuration, we measured the average per-iteration computational time and the total communication volume per iteration. All experiments were repeated three times, and the results are reported as mean values (the standard deviations were negligible and are omitted for clarity).

As can be seen in Table 7, both the computational time and the communication volume grow linearly with the number of UAVs. This linearity is expected because each UAV independently performs encryption and decryption, and the homomorphic operations (additions and plaintext–ciphertext multiplications) scale linearly with N. A linear fit to the data yields approximately 1.13 ms per additional UAV for the total time and 326.3 KB per additional UAV for the communication. Even for the largest tested swarm ($N = 40$), the per-iteration time remains under 200 ms and the communication load stays below 52 MB, which is well within the capabilities of modern wireless networks (e.g., Wi-Fi 6 or 5G) and typical control loop requirements.

Fig. 11 displays the objective function value over the 50 iterations for the four swarm sizes. All curves exhibit rapid convergence: within the first 10 iterations, the objective value drops sharply and stabilizes at a level close to zero for all tested swarm sizes, indicating that the UAVs



(a) Relative gradient error



(b) Objective function error

Fig. 9. Evolution of (a) relative gradient error and (b) objective function error over iterations($N = 10$, $t = 3$).

successfully achieve formation in every case. This demonstrates that the convergence speed and final accuracy are only minimally affected by the number of UAVs. By appropriately selecting the encryption parameters, the noise introduced by homomorphic operations remains well controlled even as the swarm size increases, ensuring that the optimization dynamics are preserved.

The linear scaling of computational cost with N together with the maintained convergence performance confirms that the proposed framework is highly scalable. It can effectively support large-scale UAV swarms with high efficiency and precision, making it suitable for practical deployments in UAV swarm scenarios.

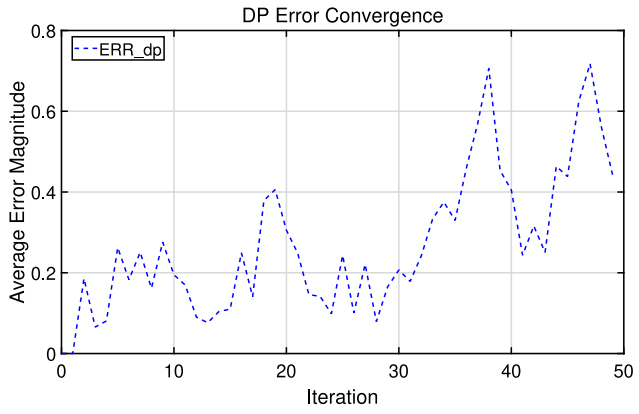
5.9. Cryptographic security verification

This section presents empirical validation of the semantic security properties of the CKKS encryption scheme. Through systematic ciphertext analysis, two fundamental security guarantees are demonstrated: (1) ciphertext indistinguishability, where ciphertexts corresponding to similar or related plaintexts exhibit no statistically discernible patterns or correlations; and (2) probabilistic encryption, wherein repeated encryption of the same plaintext under the same public key yields distinct ciphertexts. These properties collectively ensure that adversaries cannot infer meaningful information from encrypted data, thereby establishing the cryptographic foundation for secure distributed optimization.

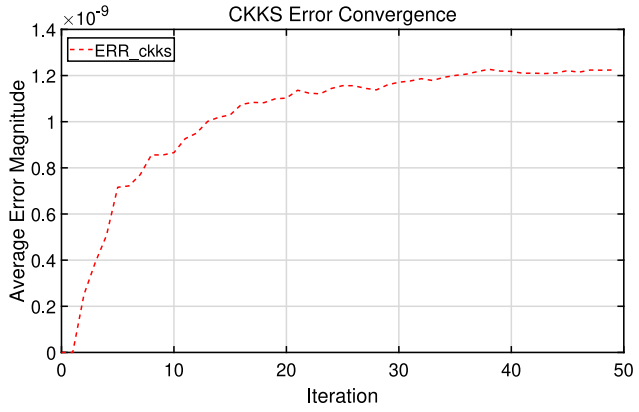
To comprehensively verify the security properties of the CKKS-encrypted distributed optimization system, two complementary ciphertext analyses are conducted. Table 8 lists the ciphertext evolution of node 2's state information for the first two and last rounds of the 50-round optimization process, showing the first 8 coefficients of the CT[0] and CT[1] polynomials. This longitudinal analysis demonstrates how ciphertext representations transform unpredictably even when plaintext states evolve gradually (from $-2 - 1i$ to $4.066 + 2.917i$), preventing attackers from deducing optimization trajectories or intermediate states.

Table 8
Ciphertext evolution of Node 2 state information.

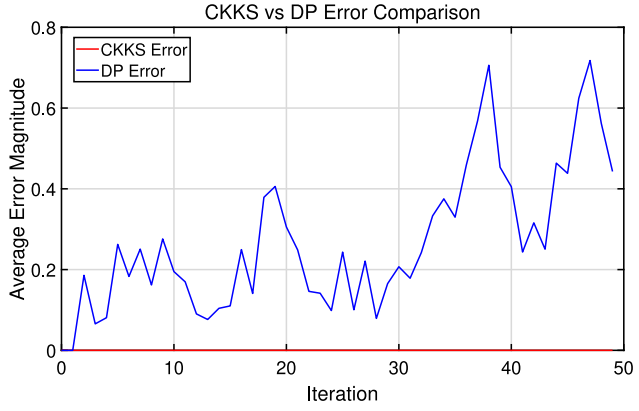
CT	Index	Round 1	Round 2	...	Round 50
		−2−1j	*4.718 + 1.412j*	...	*4.066 + 2.917j*
CT[0]	0	18 456 491	−20 695 497	...	−20 035 227
	1	92 194 680	11 122 334	...	17 870 008
	2	−21 076 739	−14 603 252	...	13 500 759
	3	11 295 189	−169 320 514	...	−14 849 837
	4	−70 222 967	19 988 907	...	15 457 180
	5	58 514 941	11 919 903	...	94 581 477
	6	25 198 803	53 257 435	...	−14 298 626
	7	−89 787 174	12 841 841	...	−16 282 436
CT[1]	0	23 137 771	99 644 982	...	68 478 845
	1	10 610 624	14 886 946	...	66 107 969
	2	15 030 480	−20 394 282	...	22 890 235
	3	13 212 096	19 595 465	...	−13 664 078
	4	−20 610 317	46 387 546	...	−20 348 644
	5	19 589 341	20 451 050	...	82 390 406
	6	−80 081 838	40 537 535	...	−17 746 204
	7	−19 063 590	20 707 699	...	42 059 303



(a) Differential privacy error



(b) CKKS encryption error



(c) Comprehensive comparison

Fig. 10. Global error comparison of privacy-preserving schemes($N = 10$, $t = 3$): (a) Differential privacy (b) CKKS encryption (c) Comprehensive comparison.

Complementing this, Table 9 provides decisive evidence of probabilistic encryption through dual encryptions of identical plaintext values at key states, showing completely unrelated ciphertext representations (e.g., $-2-1i$ produces CT[0] coefficients 18,456,491 vs. 51,278,810) despite identical inputs. The empirical evidence presented in Tables 8 and 9 therefore serves as a practical corroboration of the theoretical security properties formalized in Section 4.2. Together, these analyses confirm that neither longitudinal ciphertext sequences nor repeated encryptions reveal plaintext information, satisfying the ciphertext indistinguishability requirements for semantic security in distributed optimization systems.

Table 9
Double encryption of identical plaintext values.

Plaintext	Enc.	CT[0]	CT[1]
−2−1j	1st	18, 456, 491 92, 194, 680	23, 137, 771 10, 610, 624
	2nd	51, 27, 810 −17, 925, 894	−19, 252, 991 −18, 189, 612
4.718 + 1.412j	1st	−20, 695, 497 11,122,334	99, 644, 982 14, 886, 946
	2nd	34, 251, 309 43, 605, 071	12, 959, 992 −12, 900, 912
4.066 + 2.917j	1st	−20, 035, 227 17, 870, 008	68, 478, 845 66, 107, 969
	2nd	16, 250, 143 71, 951, 934	94, 588, 565 78, 441, 302

6. Conclusion

This paper studies privacy-preserving distributed optimization for multi-UAV cooperative positioning. A threshold CKKS-based framework is developed to integrate homomorphic encryption with distributed gradient-based optimization while preserving the decentralized interaction structure of multi-agent systems. The framework enables UAV agents to exchange encrypted states and perform secure gradient computation without revealing private information. To support efficient encrypted optimization, a noise-aware homomorphic computation structure is designed and the impact of CKKS approximation noise on the optimization process is theoretically characterized. The analysis shows that the encrypted projected gradient algorithm converges to a neighborhood of the optimal solution with bounded error. In addition, a threshold-based decentralized decryption mechanism is incorporated to eliminate the single point of trust and enhance resistance to collusion under the semi-honest adversary model. Experimental results demonstrate that the proposed method achieves convergence behavior comparable to plaintext optimization while preserving data privacy in multi-UAV systems. While the proposed framework provides strong privacy guarantees, its practical deployment in safety-critical UAV systems must account for the latency introduced by homomorphic operations, which could impact real-time tasks such as collision avoidance. Future work will focus on optimizing parameter selection, implementing asynchronous updates, and designing safety fallback mechanisms to ensure that privacy preservation does not compromise system responsiveness and operational safety.

CRediT authorship contribution statement

Zhongyuan Zhao: Writing – review & editing, Visualization, Methodology, Investigation, Conceptualization. **Jihua Fang:** Writing – original draft, Validation, Investigation, Conceptualization. **Fu Zhang:** Writing – review & editing, Software, Data curation. **Shuai Chen:** Writing – review & editing, Software, Data curation. **Feng Xie:** Writing – review & editing, Investigation, Conceptualization. **Hao Xu:** Writing – review & editing, Writing – original draft, Investigation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant U23B2061, in part by the Project Supported by the Open Fund under Grant WDZC20245250409.

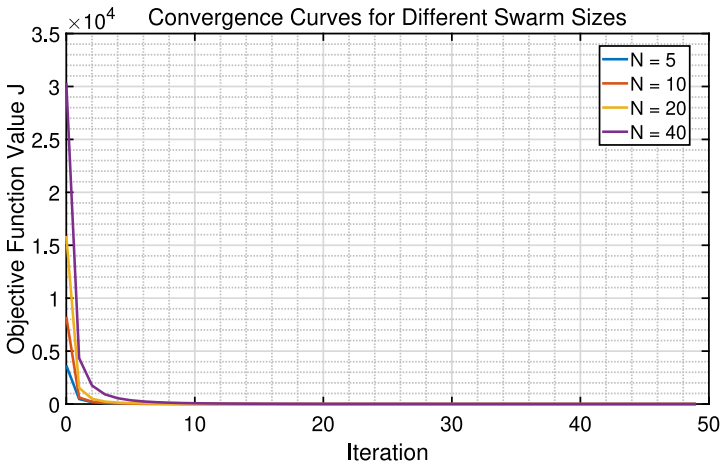


Fig. 11. Convergence curves of the objective function for different swarm sizes.

Data availability

No data was used for the research described in the article.

References

[1] R. Beard, T. McLain, M. Goodrich, E. Anderson, Coordinated target assignment and intercept for unmanned air vehicles, IEEE Trans. Robot. Autom. 18 (6) (2002) 911–922.

[2] H. Zhang, H. Ma, B.W. Mersha, X. Zhang, Y. Jin, Distributed cooperative search method for multi-UAV with unstable communications, Appl. Soft Comput. 148 (2023) 110592.

[3] T. Liu, D. Han, Y. Lin, K. Liu, Distributed multi-UAV trajectory optimization over directed networks, J. Franklin Inst. 358 (10) (2021) 5470–5487.

[4] S.S. Stanković, M.S. Stanković, D.M. Stipanović, Consensus based overlapping decentralized estimation with missing observations and communication faults, IFAC Proc. Vol. 41 (2) (2008) 9338–9343, 17th IFAC World Congress.

[5] Y. Li, W. Chen, B. Fu, S. Liu, L. Hao, Z. Wu, A distributed cooperative dynamic target search method for multi-UAV systems in complex adversarial environments, IEEE Internet Things J. 12 (18) (2025) 38155–38171.

[6] Y. Wang, K. Li, Y. Han, X. Yan, Distributed multi-UAV cooperation for dynamic target tracking optimized by an SAQPSO algorithm, ISA Trans. 129 (2022) 230–242.

[7] J.M. Such, A. Espinosa, A. García-Fornes, A survey of privacy in multi-agent systems, Knowl. Eng. Rev. 29 (3) (2014) 314–344.

[8] Q. Luo, S. Liu, L. Wang, E. Tian, Privacy-preserved distributed optimization for multi-agent systems with antagonistic interactions, IEEE Trans. Circuits Syst. I. Regul. Pap. 70 (3) (2023) 1350–1360.

[9] S.-S. Liu, M.-F. Ge, Z.-W. Liu, Multi-UAV formation control based on distributed model predictive control*, in: 2022 IEEE International Conference on Cyborg and Bionic Systems, CBS, 2023, pp. 292–297.

[10] Y. Mekdad, A. Aris, L. Babun, A.E. Fergougui, M. Conti, R. Lazzeretti, A.S. Uluagac, A survey on security and privacy issues of UAVs, Comput. Netw. 224 (2023) 109626.

[11] B. Niu, Y. Wang, J. Liu, G.X.-G. Yue, Path planning for unmanned aerial vehicles in complex environment based on an improved continuous ant colony optimisation, Comput. Electr. Eng. 123 (2025) 110034.

[12] Y. Chan, K.K. Ng, T. Wang, K. Hon, C.-H. Liu, Near time-optimal trajectory optimisation for drones in last-mile delivery using spatial reformulation approach, Transp. Res. C 171 (2025) 104986.

[13] Q. Cheng, Z. Zhang, Y. Du, X. Zhao, Hybrid metaheuristic-driven 3D path planning for UAVs in complex urban environments: a multi-objective fusion framework, Geocarto Int. 40 (1) (2025) 2538099.

[14] J. Shi, S. Zhou, X. Man, C. Li, F. Guo, Trajectory planning and tracking for UAVs with deep reinforcement learning and adaptive nonlinear MPC, Expert Syst. Appl. 303 (2026) 130649.

[15] J. Guo, J. Qi, M. Wang, C. Wu, Y. Ping, S. Li, J. Jin, Distributed cooperative obstacle avoidance and formation reconfiguration for multiple quadrotors: Theory and experiment, Aerosp. Sci. Technol. 136 (2023) 108218.

[16] M. Braquet, E. Bakolas, Greedy decentralized auction-based task allocation for multi-agent systems**This work was supported in part by a fellowship of the Belgian American educational foundation, IFAC-PapersOnLine 54 (20) (2021) 675–680, Modeling, Estimation and Control Conference MECC 2021.

[17] M. Schranz, M. Umlauft, M. Sende, W. Elmenreich, Swarm robotic behaviors and current applications, Front. Robotics AI Volume 7 - 2020 (2020).

[18] D. Wang, Z. Wang, M. Chen, W. Wang, Distributed optimization for multi-agent systems with constraints set and communication time-delay over a directed graph, Inform. Sci. 438 (2018) 1–14.

[19] C. Dwork, Differential privacy, in: M. Bugliesi, B. Preneel, V. Sassone, I. Wegener (Eds.), Automata, Languages and Programming, Springer Berlin Heidelberg, Berlin, Heidelberg, 2006, pp. 1–12.

[20] S. Han, U. Topcu, G.J. Pappas, Differentially private distributed constrained optimization, IEEE Trans. Autom. Control 62 (1) (2017) 50–64.

[21] P. Paillier, Public-key cryptosystems based on composite degree residuosity classes, in: J. Stern (Ed.), Advances in Cryptology — EUROCRYPT '99, Springer Berlin Heidelberg, Berlin, Heidelberg, 1999, pp. 223–238.

[22] Z. Brakerski, C. Gentry, V. Vaikuntanathan, (Leveled) fully homomorphic encryption without bootstrapping, ACM Trans. Comput. Theory 6 (3) (2014).

[23] J.H. Cheon, A. Kim, M. Kim, Y. Song, Homomorphic encryption for arithmetic of approximate numbers, in: Advances in Cryptology – ASIACRYPT 2017, in: Lecture Notes in Computer Science, vol. 10624, Springer, 2017, pp. 409–437.

[24] D. Boneh, R. Gennaro, S. Goldfeder, A. Jain, S. Kim, P.M.R. Rasmussen, A. Sahai, Threshold cryptosystems from threshold fully homomorphic encryption, in: H. Shacham, A. Boldyreva (Eds.), Advances in Cryptology – CRYPTO 2018, Springer International Publishing, Cham, 2018, pp. 565–596.

[25] H. Tian, Y. Wen, F. Zhang, Y. Shao, B. Li, Lattice based distributed threshold additive homomorphic encryption with application in federated learning, Comput. Stand. Interfaces 87 (2024) 103765.

[26] X. Wan, H. Lin, M. Wang, W. Shen, Hierarchical threshold multi-key fully homomorphic encryption, J. Inf. Secur. Appl. 89 (2025) 103919.

[27] F. Li, B. Luo, P. Liu, Secure information aggregation for smart grids using homomorphic encryption, in: 2010 First IEEE International Conference on Smart Grid Communications, 2010, pp. 327–332.

[28] S.M. Rostamkolaei Motlagh, C. Pahl, H.R. Barzegar, N. El Ioini, A technology review of zero knowledge proof techniques, in: Proceedings of the 10th International Conference on Internet of Things, Big Data and Security - IoTBDS, INSTICC, SciTePress, 2025, pp. 245–253.

[29] C. Paar, J. Pelzl, Understanding Cryptography: A Textbook for Students and Practitioners, Springer, 2010.

[30] W. Stallings, Cryptography and Network Security: Principles and Practice, third ed., Pearson Education, 2002.

[31] V. Lyubashevsky, C. Peikert, O. Regev, On ideal lattices and learning with errors over rings, in: H. Gilbert (Ed.), Advances in Cryptology – EUROCRYPT 2010, Springer Berlin Heidelberg, Berlin, Heidelberg, 2010, pp. 1–23.

[32] P. Feldman, A practical scheme for non-interactive verifiable secret sharing, in: 28th Annual Symposium on Foundations of Computer Science, Sfcs 1987, 1987, pp. 427–438.

[33] C. Peikert, B. Waters, Lossy trapdoor functions and their applications, in: Proceedings of the Fortieth Annual ACM Symposium on Theory of Computing, STOC '08, Association for Computing Machinery, New York, NY, USA, 2008, pp. 187–196.

[34] A. Shamir, How to share a secret, Commun. ACM 22 (11) (1979) 612–613.

[35] S. Boyd, L. Vandenberghe, Convex Optimization, Cambridge University Press, 2004.